

MEMORIA DEL PROYECTO FINAL

PROGRAMACIÓN DE APLICACIONES TELEMÁTICAS

ChisPadel — Sistema de Reserva de Pistas de Pádel



Grupo 5

Ana Mei Li Ramos Flores
Martina Ortiz Delgado
Yago Méndez Fernández
Antonio Lafont Carrasco
Felicia Huynh

UNIVERSIDAD PONTIFICIA COMILLAS, ICAI
Grado en Ingeniería en Tecnologías de Telecomunicación
3.º GITT — Curso 2025/2026

Mayo de 2026

ÍNDICE

ÍNDICE	2
1. Introducción	4
1.1. Contexto y motivación del proyecto	4
1.2. Elección del dominio del problema	4
1.3. Objetivos generales del proyecto	5
1.4. Herramientas y tecnologías empleadas	6
1.4.1. Tecnologías del backend	6
1.4.2. Tecnologías del frontend	6
1.4.3. Herramientas de desarrollo y colaboración	7
2. Análisis y Planificación del Proyecto	8
2.1. Identificación y especificación de requisitos	8
2.1.1. Requisitos funcionales	8
2.1.2. Requisitos no funcionales	9
2.2. Estrategia de reparto del trabajo	11
2.3. Metodología ágil y roles del equipo	12
2.4. Organización del repositorio y estrategia de ramas	12
2.5. Planificación temporal y entregas	13
3. Desarrollo del Backend	14
3.1. Arquitectura general del backend	14
3.1.1. Capas y flujo de una petición	14
3.1.2. Organización del código en paquetes	16
3.2. Modelo de datos y entidades JPA	16
3.2.1. Entidad Usuario	16
3.2.2. Entidad Pista	17
3.2.3. Entidad Reserva	18
3.2.4. Entidad Disponibilidad	18
3.2.5. Entidad Rol y enumerados	19
3.2.6. Relaciones entre entidades y diagrama ER	19
3.3. Capa de persistencia: los repositorios	21
3.3.1. RepoUsuario	21
3.3.2. RepoPista	21
3.3.3. RepoReserva	22
3.3.4. RepoDisponibilidad y RepoRol	22
3.4. Objetos de transferencia de datos (DTOs)	22
3.4.1. RegisterRequest	22
3.4.2. LoginRequest y LoginResponse	23
3.4.3. UsuarioResponse	23
3.4.4. DisponibilidadResponse	23

3.5. Subsistema de seguridad	23
3.5.1. Configuración principal: ConfigSeguridad	24
3.5.2. El filtro BearerTokenFilter	25
3.5.3. Gestión de tokens en UsuarioService	26
3.5.4. Cifrado de contraseñas con BCrypt	27
3.5.5. Configuración para pruebas: TestSecurityConfig	27
3.6. Implementación de los endpoints de la API	28
3.6.1. Endpoints del Bloque de Pistas	28
3.6.2. Endpoints del Bloque de Disponibilidad	31
3.6.3. Endpoints del Bloque de Reservas	33
3.6.4. Endpoints del Bloque de Autenticación y Gestión de Usuarios	36
3.7. Tareas programadas y notificaciones	41
3.8. Tratamiento de errores y trazabilidad	42
3.9. Estrategia de pruebas del backend	43
3.9.1. Pruebas de integración de la capa de repositorio con @DataJpaTest	43
3.9.2. Pruebas de extremo a extremo (E2E) con @SpringBootTest	44
3.9.3. Estrategia de aislamiento mediante perfiles y @DirtiesContext	44
4. DESARROLLO DE LA INTERFAZ DE USUARIO (FRONTEND)	46
4.1. Arquitectura y Diseño Estético Global	46
4.2. Arquitectura de Integración y Ciclo de Vida del Token de Sesión	46
4.2.1. Lógica de Sincronización Asíncrona del Lado del Cliente (usuario.js)	47
4.2.2. Panel Administrativo Global y Generación Estructural de Listados (admin.js)	49
4.3. Módulo de Gestión de Pistas	50
4.3.1. Panel de Administración y Mutación de Pistas (pista.html y pista.js)	51
4.3.2. Vista de Catálogo e Información de Pistas (todosPistas.html y todosPistas.js)	52
4.4. Módulo de Disponibilidad en Tiempo Real	55
4.4.1. Consulta Dinámica y Rejilla de Estados (disponibilidad.html y disponibilidad.js)	55
4.4.2. Desglose Estructurado de Bloques Horarios (Vista de Detalle)	59
4.5. Módulo de Transacciones de Reserva	60
4.5.1. Flujo de Usuario: Creación, Visualización y Reprogramación (reserva.html y reservas.js)	60
4.5.2. Flujo de Administración: Panel de Supervisión Total de Reservas	62
5. CONCLUSIONES Y TRABAJO FUTURO	65
5.1. Conclusiones y Consolidación de Conocimientos	65

1. Introducción

1.1. Contexto y motivación del proyecto

A lo largo de la asignatura y del proyecto final se ha podido experimentar cómo, el desarrollo de aplicaciones exige la integración de forma coherente una multiplicidad de capas tecnológicas heterogéneas: la lógica de negocio del servidor, la persistencia de datos en sistemas gestores de bases de datos, los mecanismos de seguridad y autenticación, la comunicación entre componentes distribuidos mediante protocolos estandarizados y, finalmente, la interfaz de usuario que el cliente percibe y con la que interactúa.

La presente memoria documenta de forma exhaustiva el diseño, la implementación, la integración y la validación de ChisPadel, una aplicación web completa para la gestión de reservas de pistas de pádel, desarrollada por nuestro grupo, el Grupo 5, a lo largo del curso de Programación de Aplicaciones Telemáticas. Este proyecto representa la culminación del aprendizaje adquirido en la asignatura y materializa los conocimientos teóricos y prácticos trabajados durante el curso.

El objetivo del proyecto no se limita únicamente a programar el código fuente de la aplicación, sino que abarca el ciclo completo de desarrollo de aplicaciones: desde el análisis de requisitos y la planificación del trabajo, pasando por el diseño de la arquitectura y la implementación iterativa mediante metodologías ágiles, hasta la realización de pruebas automatizadas que garantizan la estabilidad, la corrección funcional y la seguridad del sistema, cuidando aspectos relevantes como el control de versiones o la integración continua.

De manera organizada y hablada, documentando de forma sistemática cada decisión técnica relevante, se ha podido experimentar de forma directa las distintas etapas que componen el ciclo de vida completo del software en un entorno que simula las condiciones del trabajo profesional real. El proyecto ha permitido al equipo enfrentarse a desafíos auténticos de coordinación entre múltiples desarrolladores, de integración de componentes desarrollados de forma paralela, de control de calidad mediante revisiones de código y de gestión de conflictos técnicos y de priorización de tareas. Estas competencias, que complementan los conocimientos puramente técnicos, resultan indispensables para el ejercicio profesional de la ingeniería del software y difícilmente se adquieren fuera de una experiencia práctica de este tipo.

1.2. Elección del dominio del problema

El tema sobre el que se construyó el proyecto no fue libre, sino que fue el mismo para todos los grupos; sin embargo, consideramos que la elección de un sistema de gestión de reservas de instalaciones deportivas —concretamente, de pistas de pádel— como caso de estudio no es casual ni arbitraria. Este tipo de aplicaciones representa un dominio de complejidad media que resulta ideal para un proyecto académico de esta envergadura, ya que permite ejercitar un amplio abanico de competencias técnicas sin caer en la trivialidad ni tampoco en una complejidad excesiva que haría inmanejable el proyecto en el tiempo disponible.

El dominio elegido nos obligó a afrontar problemas técnicos de interés real que son representativos de las aplicaciones de gestión empresarial más habituales en la industria. Entre estos problemas destacan, en primer lugar, el manejo de conflictos de concurrencia temporal: el sistema debe garantizar que no puedan existir dos reservas que se solapen sobre la misma pista, lo que exige implementar mecanismos de detección de colisiones en intervalos temporales y resolver correctamente los casos límite. En segundo lugar, la gestión de permisos diferenciados

por rol de usuario obliga a implementar un sistema de autenticación y autorización que distinga con precisión entre usuarios estándar y administradores, controlando el acceso a recursos en función de la identidad del solicitante.

En tercer lugar, la programación de tareas automáticas que se ejecuten de forma periódica — como el envío de correos de recordatorio— introduce el concepto de procesos en segundo plano que operan de forma independiente del flujo de peticiones HTTP. Por último, la persistencia de datos con relaciones complejas entre entidades —usuarios que realizan reservas sobre pistas— exige diseñar un modelo de datos relacional coherente y mapearlo correctamente a una base de datos mediante un framework de persistencia.

Además de que el dominio elegido presenta una ventaja clara y es que se trata de un problema comprensible de forma intuitiva para cualquier persona que haya practicado deporte en instalaciones compartidas, lo que facilitó la implementación de este. Esta claridad conceptual permitió centrar el esfuerzo en los aspectos técnicos sin verse obstaculizado por la comprensión del dominio de negocio.

1.3. Objetivos generales del proyecto

El **objetivo principal** del proyecto es el diseño, la construcción y la validación de una aplicación telemática que permita a un club de pádel gestionar de forma centralizada y automatizada sus pistas, sus usuarios y las reservas que estos realizan. La aplicación resuelve un problema concreto y cotidiano que afecta a numerosos centros deportivos: sustituir la gestión manual de reservas —propensa a errores humanos, a solapamientos de franjas horarias, a dobles ocupaciones accidentales y a pérdidas de información— por un sistema informático que automatiza la comprobación de disponibilidad, controla de forma estricta los permisos de cada tipo de usuario, mantiene un registro persistente de todas las operaciones y notifica de forma proactiva a los jugadores sobre los eventos relevantes que les conciernen.

ChisPadel se estructura en torno a una arquitectura **cliente-servidor** clásica. El servidor expone una API REST desarrollada íntegramente con Spring Boot, el framework de referencia del ecosistema Java para la construcción de aplicaciones web y de microservicios. La persistencia de los datos se delega en una base de datos relacional H2, gestionada de forma transparente mediante Spring Data JPA, que abstrae las operaciones de acceso a datos y permite trabajar con objetos Java en lugar de con sentencias SQL. El cliente, por su parte, consiste en una interfaz web construida con las tecnologías estándar de la plataforma web —HTML5 para la estructura semántica del contenido, CSS3 para la presentación visual y JavaScript para la lógica de interacción—, sin recurrir a frameworks de frontend de terceros, de modo que el equipo comprende en profundidad cada línea de código y cada decisión técnica.

El sistema distingue dos perfiles o **roles de usuario** claramente diferenciados: el usuario estándar, identificado con el rol USER, que puede registrarse, autenticarse, consultar la disponibilidad de las pistas, crear reservas, consultar sus propias reservas y modificarlas o cancelarlas; y el administrador, identificado con el rol ADMIN, que hereda todas las capacidades del usuario estándar y añade, además, la posibilidad de gestionar el catálogo de pistas (crear, modificar y eliminar pistas), consultar la lista completa de usuarios registrados en el sistema y acceder a la totalidad de las reservas, independientemente de quién las haya realizado. Esta separación de perfiles y de permisos es una característica fundamental de cualquier aplicación de gestión empresarial y su correcta implementación constituye uno de los aspectos más delicados del proyecto.

1.4. Herramientas y tecnologías empleadas

Con el objetivo de aproximar el proyecto a un entorno de producción real y de familiarizar al equipo con las herramientas profesionales más empleadas en la industria del desarrollo de software, se han utilizado las siguientes tecnologías, las cuales fueron recomendadas por el equipo docente.

1.4.1. Tecnologías del backend

- **Java 21:** lenguaje de programación principal del backend. Se ha optado por la versión 21, una versión LTS (Long-Term Support) que incorpora las características modernas del lenguaje —como records, pattern matching y text blocks— al tiempo que garantiza estabilidad y soporte a largo plazo.
- **Spring Boot 3:** framework que proporciona el contenedor de inyección de dependencias, la configuración automática de componentes y la integración transparente con el resto de los módulos del ecosistema Spring, entre otros. Spring Boot simplifica enormemente la configuración inicial de un proyecto y permite al equipo centrarse en la lógica de negocio en lugar de en la infraestructura.
- **Spring Data JPA:** módulo de Spring que proporciona la capa de persistencia del sistema, abstrayendo el acceso a la base de datos mediante repositorios y permitiendo trabajar con entidades Java en lugar de con sentencias SQL. Hibernate actúa como proveedor de JPA, traduciendo las operaciones sobre objetos en las consultas SQL correspondientes.
- **Base de datos H2:** sistema gestor de base de datos relacional embebido, escrito en Java, que puede ejecutarse tanto en memoria como persistiendo los datos en fichero. Para el proyecto se ha configurado en modo fichero para conservar los datos entre arranques. H2 incorpora además una consola web de administración que permite inspeccionar el esquema y los datos durante el desarrollo.
- **Spring Security:** módulo de seguridad que proporciona los mecanismos de autenticación, autorización, protección contra ataques comunes (CSRF, XSS) y configuración de CORS. La arquitectura de Spring Security se basa en una cadena de filtros que interceptan las peticiones entrantes y aplican las políticas de seguridad configuradas.
- **Spring Boot Mail:** módulo que abstrae el envío de correos electrónicos mediante el protocolo SMTP, empleado por las tareas programadas para notificar a los usuarios.
- **JUnit 5 y Spring Boot Test:** frameworks de pruebas automatizadas. JUnit 5 proporciona las anotaciones y las aserciones básicas, mientras que Spring Boot Test aporta las anotaciones específicas para levantar el contexto de Spring en distintas configuraciones según el tipo de prueba.
- **BCrypt:** algoritmo de hashing de contraseñas con sal incorporada, empleado para cifrar las contraseñas de los usuarios antes de persistirlas en la base de datos.

1.4.2. Tecnologías del frontend

- **HTML5:** lenguaje de marcado que define la estructura semántica de las páginas web. Se han empleado las etiquetas semánticas modernas (header, nav, main, section, article, footer) para mejorar la accesibilidad y el SEO.
- **CSS3:** lenguaje de hojas de estilo que controla la presentación visual. Se han empleado características modernas como custom properties (variables CSS), flexbox, grid y media queries para lograr un diseño responsivo que se adapta a distintos tamaños de pantalla.

- **JavaScript ES6+:** lenguaje de programación del lado del cliente, empleando las características modernas del estándar ECMAScript 6 y posteriores: funciones flecha, async/await, desestructuración, template literals y módulos.
- **API Fetch:** interfaz estándar del navegador para realizar peticiones HTTP asíncronas, que sustituye a XMLHttpRequest con una API más moderna y basada en promesas.
- **localStorage:** API del navegador que permite almacenar datos de forma persistente en el cliente, empleada para mantener el estado de la sesión del usuario entre recargas de página.

1.4.3. Herramientas de desarrollo y colaboración

- **IntelliJ IDEA:** entorno de desarrollo integrado (IDE) empleado para la programación del backend. IntelliJ ofrece soporte avanzado para Java, Spring Boot y Maven, incluyendo autocompletado inteligente, refactorización automática, depuración integrada y ejecución de tests.
- **Visual Studio Code:** editor de código ligero empleado para el desarrollo del frontend. VS Code proporciona extensiones para HTML, CSS y JavaScript, así como herramientas de depuración para el navegador y servidores de desarrollo en vivo.
- **Git y GitHub:** sistema de control de versiones distribuido y plataforma de alojamiento de repositorios. Git permite al equipo trabajar de forma paralela sobre ramas independientes y fusionar los cambios de forma controlada. GitHub aporta además funcionalidades de revisión de código, gestión de incidencias y automatización mediante GitHub Actions.
- **GitHub Actions:** servicio de integración y despliegue continuos que ejecuta automáticamente la batería de pruebas cada vez que se incorpora código al repositorio, detectando regresiones de forma temprana.
- **GitHub Pages:** servicio de alojamiento web estático que publica automáticamente el frontend de la aplicación directamente desde el repositorio.
- **Postman:** herramienta de prueba manual de APIs REST que permite definir colecciones de peticiones con sus parámetros, cabeceras y cuerpos, y ejecutarlas de forma interactiva. El equipo ha creado una colección con los veintidós endpoints del proyecto para facilitar las pruebas durante el desarrollo.
- **Maven:** herramienta de gestión de dependencias y de construcción de proyectos Java. Maven descarga automáticamente las librerías necesarias, compila el código, ejecuta las pruebas y genera los artefactos desplegables.

2. Análisis y Planificación del Proyecto

Antes de abordar la implementación técnica de ChisPadel, el equipo dedicó una fase inicial de análisis a comprender en profundidad los requisitos de la aplicación, a diseñar una estrategia de reparto del trabajo que equilibrara la carga entre los cinco miembros y a organizar la colaboración mediante una metodología ágil que permitiera el desarrollo iterativo e incremental. Este capítulo documenta las decisiones tomadas en dicha fase, que han condicionado de forma determinante todo el desarrollo posterior.

2.1. Identificación y especificación de requisitos

2.1.1. Requisitos funcionales

Se describen las operaciones que el sistema debe ser capaz de realizar y las funcionalidades que debe ofrecer a sus usuarios.

Gestión de usuarios:

- El sistema debe permitir el registro de nuevos usuarios mediante un formulario que recoja sus datos personales: nombre, apellidos, correo electrónico, teléfono y contraseña.
- El sistema debe validar que el correo electrónico tenga un formato válido y que no exista ya otro usuario registrado con el mismo correo, rechazando el registro en caso contrario con un mensaje de error descriptivo.
- Las contraseñas de los usuarios deben almacenarse cifradas mediante un algoritmo de hashing seguro, de modo que nunca se persistan en texto plano en la base de datos.
- El sistema debe proporcionar un mecanismo de autenticación que permita a los usuarios iniciar sesión proporcionando su correo y contraseña, y que emita un token de sesión válido durante un periodo de tiempo limitado.
- Los usuarios autenticados deben poder consultar y modificar sus propios datos personales, excepto aquellos que el sistema considere inmutables por razones de integridad (identificador, rol, fecha de registro).
- El sistema debe ofrecer un mecanismo de cierre de sesión que invalide el token activo, de modo que no pueda ser reutilizado tras el logout.

Gestión de pistas:

- El sistema debe mantener un catálogo de pistas de pádel, cada una con su nombre único, su ubicación, su precio por hora y su estado de activación.
- Los usuarios autenticados deben poder consultar el listado completo de pistas disponibles y el detalle de cada una de ellas.
- Únicamente los usuarios con rol de administrador deben poder crear nuevas pistas, modificar los datos de las pistas existentes y eliminarlas o desactivarlas.
- El nombre de cada pista debe ser único en el sistema, rechazándose la creación o modificación que provoque duplicados.
- No debe ser posible eliminar una pista que tenga reservas futuras asociadas, protegiéndose así la integridad referencial del sistema.

Gestión de reservas:

- Los usuarios autenticados deben poder crear reservas de pistas para una fecha y hora determinadas, especificando la duración de la reserva.
- El sistema debe validar que la pista especificada existe, que está activa y que el instante de inicio es anterior al de fin.
- El sistema debe detectar automáticamente los solapamientos temporales, rechazando con un mensaje de error cualquier reserva que se superponga, aunque sea parcialmente, con otra reserva existente sobre la misma pista.
- Los usuarios deben poder consultar el listado de sus propias reservas, con la posibilidad de filtrar por rango de fechas.
- Los usuarios deben poder reprogramar sus reservas, cambiando la fecha, la hora de inicio o la duración, respetando las mismas validaciones que en la creación.
- Los usuarios deben poder cancelar sus propias reservas. La cancelación no elimina el registro de la base de datos, sino que cambia su estado a CANCELADA.
- Los administradores deben poder consultar todas las reservas del sistema, independientemente de quién las haya realizado, con posibilidad de filtrar por fecha, por pista y por usuario.
- Los administradores deben poder reprogramar o cancelar cualquier reserva, no solo las propias.

Consulta de disponibilidad:

- El sistema debe calcular y ofrecer la disponibilidad de las pistas para una fecha determinada, mostrando las franjas horarias libres.
- Debe ser posible consultar la disponibilidad general de todas las pistas o filtrar por una pista concreta.
- El cálculo de la disponibilidad debe tener en cuenta el horario de apertura y cierre del club, las reservas existentes y el estado de activación de cada pista.

Notificaciones automáticas (Tareas programadas):

- El sistema debe enviar automáticamente, todas las madrugadas, un correo de recordatorio a cada usuario que tenga una reserva programada para ese mismo día.
- El sistema debe enviar automáticamente, el primer día de cada mes, un correo a todos los usuarios registrados informándoles de la disponibilidad de pistas y horarios para el mes entrante.

2.1.2. Requisitos no funcionales

Se describen las características de calidad que debe poseer el sistema y las restricciones bajo las que debe operar.

Arquitectura y diseño:

- La API debe seguir el estilo arquitectónico REST, empleando los métodos HTTP de forma semántica y estructurando las URI de forma jerárquica y descriptiva.

- El backend debe organizarse siguiendo el patrón Modelo-Vista-Controlador (MVC) con una clara separación de responsabilidades entre las capas de presentación (controlador), lógica de negocio (servicio) y persistencia (repositorio).
- El modelo de datos debe estar normalizado al menos hasta la tercera forma normal para evitar redundancias y anomalías de actualización.

Seguridad:

- El acceso a cualquier recurso protegido debe estar condicionado a la presentación de credenciales válidas. La ausencia de autenticación debe producir un código de respuesta 401 Unauthorized.
- El sistema debe distinguir con precisión entre la ausencia de autenticación (401) y la falta de permisos para la operación solicitada (403 Forbidden), proporcionando mensajes de error claros en cada caso.
- Las contraseñas no deben almacenarse nunca en texto plano, sino que deben cifrarse mediante un algoritmo de hashing con sal antes de su persistencia.
- El sistema debe implementar un mecanismo de tokens de sesión que caduquen tras un periodo de inactividad, obligando al usuario a volver a autenticarse.
- La API debe configurar adecuadamente las cabeceras CORS para permitir las peticiones desde orígenes distintos al del servidor, habilitando así la comunicación con el frontend.

Códigos de estado HTTP:

- La API debe devolver en cada caso el código de estado HTTP que describe con mayor precisión el resultado de la operación, siguiendo el estándar RFC 7231.
- Las operaciones de creación satisfactorias deben devolver 201 Created.
- Las operaciones de modificación satisfactorias deben devolver 200 OK.
- Las operaciones de eliminación satisfactorias que no devuelven contenido deben responder con 204 No Content.
- Las peticiones con datos inválidos o mal formados deben rechazarse con 400 Bad Request.
- Los conflictos de negocio (duplicados, solapamientos) deben notificarse con 409 Conflict.
- Las peticiones sobre recursos inexistentes deben devolver 404 Not Found.

Trazabilidad y observabilidad:

- La aplicación debe registrar trazas en distintos niveles de severidad (info, debug, error) que permitan seguir el flujo de ejecución y diagnosticar problemas.
- El sistema debe exponer un endpoint de healthcheck que permita verificar que el servicio está en ejecución y respondiendo correctamente, facilitando su integración en pipelines de CI/CD.

Calidad y testing:

- El sistema debe estar cubierto por una batería de pruebas automatizadas que verifiquen su correcto funcionamiento tanto a nivel de integración como de extremo a extremo.
- Las pruebas deben ejecutarse en un entorno aislado, con su propia configuración de seguridad y su propia base de datos en memoria, de modo que no interfieran con el sistema en producción.
- Cada prueba debe ser independiente de las demás, garantizando que el orden de ejecución no afecta a los resultados.

2.2. Estrategia de reparto del trabajo

Uno de los desafíos clave del proyecto fue diseñar una distribución de tareas equilibrada entre los cinco miembros, que minimizara conflictos de código y fomentara la especialización autónoma. Para ello, se optó por un reparto por área funcional o entidad, asignando a cada desarrollador la responsabilidad exclusiva de un bloque completo de endpoints relacionados conceptualmente, en lugar de dividir el trabajo por capas tecnológicas.

Esta estrategia facilitó que cada integrante dominara las reglas de negocio de su área y redujo drásticamente las colisiones en el repositorio al operar sobre entidades independientes. Además, optimizó el proceso de pruebas al permitir validar los desarrollos de forma aislada sin depender completamente del avance de los demás compañeros, replicando fielmente la organización por dominios o microservicios de los entornos laborales profesionales.

Finalmente, el reparto se niveló evaluando tanto el volumen de endpoints como su complejidad técnica intrínseca. De esta manera, se equilibraron bloques más compactos, pero con mayor dificultad algorítmica y de seguridad —como la autenticación, que requiere la gestión estricta de tokens y control de errores HTTP 401/403, o el cálculo de disponibilidad de pistas— con el desarrollo de operaciones CRUD convencionales.

Miembro	Área asignada	Endpoints principales	Mezcla CRUD
Felicia Huynh	Pistas (courts)	POST, GET (lista), GET (detalle), PATCH, DELETE sobre /courts	CRUD completo
Ana Mei Li Ramos	Autenticación y detalle de usuario	POST /auth/register, /auth/login, /auth/logout; GET /auth/me, /users/{id}	2×POST, 2×GET, 1×DELETE lógico
Martina Ortiz	Gestión de usuarios y healthcheck	GET /users, PATCH /users/{id}, GET /health	1×GET, 1×PATCH, 1×healthcheck
Antonio Lafont	Reservas (reservations)	POST, GET (detalle), PATCH, DELETE, GET /admin/reservations	CRUD casi completo
Yago Méndez	Disponibilidad y listados del usuario	GET /availability, GET /courts/{id}/availability, GET /reservations	3×GET con cálculo

Cada miembro asumió, además de la implementación de los endpoints, la responsabilidad completa sobre las entidades, servicios, repositorios y pruebas asociadas a su área. Este

enfoque integral garantiza que quien desarrolla una funcionalidad es también quien la valida mediante tests, lo que favorece la calidad del código y la detección temprana de errores.

2.3. Metodología ágil y roles del equipo

Para simular el modo en que se diseña y produce software en el mundo profesional, el equipo adoptó desde el inicio una metodología ágil inspirada en Scrum. Esta estructura permitió mantener una visión compartida del estado del proyecto, detectar bloqueos de forma temprana y redistribuir tareas cuando fue necesario.

Para el desarrollo del proyecto se propuso la división en cuatro roles:

- **Scrum Master:** responsable de velar por que el equipo siga la metodología ágil, de organizar y moderar las reuniones de seguimiento, de eliminar impedimentos que bloqueen el progreso del equipo y de facilitar la comunicación entre los miembros.
- **Program Manager (Product Owner):** encargado de validar que el trabajo desarrollado responde correctamente a las peticiones planteadas en los requisitos y tiene sentido funcional desde el punto de vista del usuario final.
- **Analista/Desarrollador:** rol centrado en el desarrollo del código que materializa los requisitos. Los analistas implementan las funcionalidades asignadas, documentan el código, realizan revisiones de código de sus compañeros y participan en las decisiones técnicas del equipo.
- **Release Manager / QA (Quality Assurance):** responsable de realizar las integraciones de código (merges de ramas de desarrollo en la rama principal), de garantizar que todo el código se encuentra en orden y supera las pruebas de calidad antes de cada entrega, de ejecutar la batería de tests completa y de preparar los artefactos desplegables.

La repartición de los roles descritos no fue estricta, ya que cada miembro del grupo adoptó cada rol en función de la necesidad, sin tener ninguno fijo. Esto dio dinamismo y versatilidad al trabajo grupal.

Para la coordinación del trabajo y llevar un seguimiento de qué modificaciones se realizaban en cada rama, cada componente lo notificaba por el grupo de Whatsapp que se creó o lo reflejaba en un fichero README.

2.4. Organización del repositorio y estrategia de ramas

El repositorio Git del proyecto se estructuró siguiendo el modelo de flujo de trabajo Git Flow. Este modelo distingue varios tipos de ramas con propósitos diferentes, lo que proporciona un marco claro para la colaboración en equipo y facilita la gestión de versiones.

El repositorio mantiene dos ramas principales que nunca se eliminan:

- **main:** rama que aloja la última versión estable del proyecto, aquella que ha sido entregada oficialmente y que se considera lista para producción. Esta rama solo recibe commits mediante merges desde otras ramas, nunca de forma directa, y su historial debe ser siempre compilable y pasar todas las pruebas.
- **develops:** ramas de integración continua donde se reúnen todos los desarrollos en curso del equipo bajo el nombre de “nombre_branch”. Estas ramas representan el estado actual del próximo release.

2.5. Planificación temporal y entregas

El proyecto se planificó conforme al calendario de entregas establecido en la guía de la práctica, que divide el trabajo en dos grandes bloques: una primera entrega centrada en el backend y la base de datos, y una segunda entrega que incorpora el frontend y la funcionalidad completa de la aplicación. Cada uno a su vez con dos entregas intermedias, lo que permitió al equipo recibir retroalimentación temprana de los profesores y ajustar el rumbo si era necesario.

Hito	Contenido entregado	Fecha límite
Backend — Lógica	Implementación de la API REST con Spring Boot, modelo de datos y lógica de negocio sin persistencia. Pruebas unitarias de servicios.	18 de febrero de 2026
Backend — Persistencia	Integración completa con base de datos H2, migración del modelo a JPA, implementación de repositorios. Pruebas de integración y E2E.	18 de marzo de 2026
Frontend — Maquetación	Estructura HTML de todas las páginas, sistema de estilos CSS, diseño responsivo. Integración preliminar con la API.	15 de abril de 2026
Frontend — Funcionalidad	Lógica JavaScript completa, comunicación con la API mediante Fetch, gestión del estado de sesión. Aplicación completamente funcional.	17 de mayo de 2026

El desarrollo del backend ocupó las primeras seis semanas del proyecto. Durante la primera fase, el equipo se centró en implementar la estructura de clases de servicio y controlador, definir las interfaces de repositorio y desarrollar las reglas de negocio sin preocuparse aún por la persistencia real de los datos. Esta aproximación permitió validar la lógica y detectar problemas de diseño de forma temprana. En la segunda fase se integró Spring Data JPA, se mapearon las entidades a tablas de base de datos y se implementaron las consultas derivadas de los repositorios. En cada fase, el equipo implementó a su vez test para efectuar pruebas del correcto funcionamiento del proyecto.

El desarrollo del frontend ocupó las siguientes seis semanas. La primera fase se dedicó a la maquetación de todas las páginas HTML, al diseño del sistema de estilos compartido y a la creación de una primera versión estática de la interfaz que permitiera validar el diseño visual con el equipo. En la segunda fase se implementó toda la lógica JavaScript, se conectó el frontend con la API REST mediante llamadas Fetch, se gestionó el estado de sesión del usuario y se pulieron los detalles de interacción, obteniendo así una aplicación web completamente funcional de extremo a extremo.

3. Desarrollo del Backend

El backend constituye el núcleo lógico de ChisPadel: es la parte de la aplicación encargada de procesar las peticiones de los clientes, de aplicar las reglas de negocio del dominio, de gestionar la persistencia de los datos y de orquestar las interacciones entre los distintos componentes del sistema. El backend se ha construido íntegramente sobre Spring Boot, siguiendo el patrón arquitectónico Modelo-Vista-Controlador (MVC) adaptado al contexto de las API REST, donde la "vista" tradicional se sustituye por las respuestas JSON que el controlador devuelve al cliente.

Este capítulo describe de forma exhaustiva todos los aspectos del desarrollo del backend: su arquitectura de capas y el flujo que sigue una petición desde que llega al servidor hasta que se devuelve la respuesta; el modelo de datos y las entidades que lo componen; la capa de persistencia y los repositorios; el subsistema de seguridad basado en tokens; el tratamiento centralizado de errores; los veintitres endpoints de la API agrupados por la persona responsable de cada bloque; las tareas programadas de notificación automática; y, finalmente, la estrategia de pruebas que garantiza la calidad del código.

3.1. Arquitectura general del backend

La aplicación se organiza en una arquitectura de capas en la que cada componente tiene una responsabilidad bien delimitada y se comunica únicamente con las capas inmediatamente adyacentes. Esta separación de responsabilidades reporta múltiples beneficios: facilita la comprensión del código, ya que cada capa tiene un propósito claro; simplifica el mantenimiento, dado que los cambios en una capa tienen un impacto limitado en las demás; permite el testing aislado de cada componente; y posibilita la sustitución de implementaciones sin afectar al resto del sistema.

3.1.1. Capas y flujo de una petición

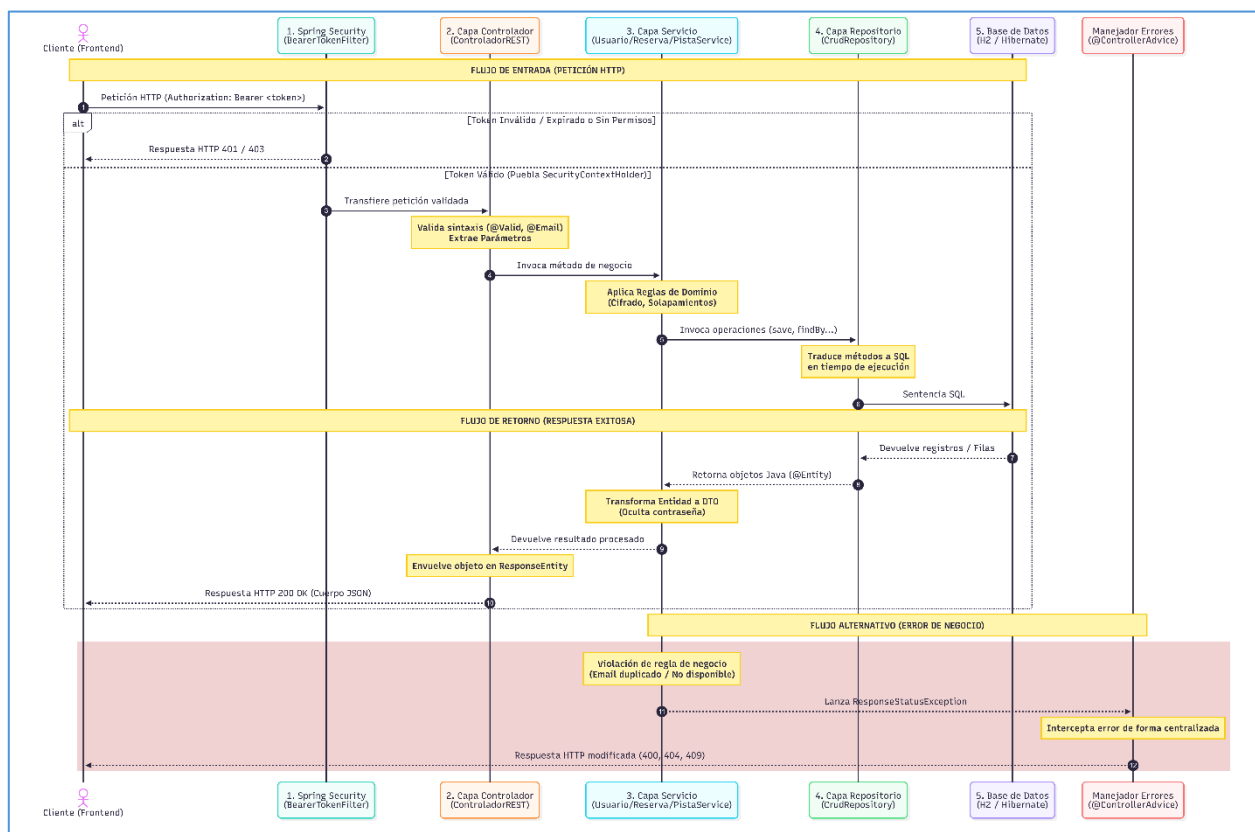
Cuando una petición HTTP llega al puerto 8080 del servidor, atraviesa secuencialmente las siguientes capas antes de producir una respuesta:

- **1. Cadena de filtros de seguridad:** antes de que la petición alcance el controlador, pasa por la cadena de filtros configurada en Spring Security. El filtro de token Bearer, implementado por el equipo como `BearerTokenFilter`, intercepta la petición, extrae el token de la cabecera `Authorization`, lo valida contra el almacén de tokens en memoria, recupera el usuario asociado de la base de datos y puebla el `SecurityContextHolder` con la autenticación correspondiente. Si el token es inválido o ha caducado, el filtro puede optar por rechazar la petición con un 401 o por dejarla pasar sin autenticación, según la configuración. Los filtros posteriores de Spring Security verifican si el endpoint solicitado requiere autenticación y, en su caso, si el usuario autenticado tiene los permisos necesarios.
- **2. Capa de controlador:** si la petición supera los filtros de seguridad, llega al método correspondiente de la clase `ControladorREST`, que actúa como único punto de entrada HTTP de la aplicación. El controlador es responsable de validar la estructura sintáctica de los datos de entrada mediante anotaciones de Bean Validation (`@Valid`, `@NotBlank`, `@Email`, etc.), de extraer los parámetros de la petición (path variables, query parameters, request body) y de invocar al método apropiado del servicio correspondiente, pasándole los datos ya validados.
- **3. Capa de servicio:** las clases de servicio (`UsuarioService`, `PistaService`, `ReservaService`, `DisponibilidadService`) concentran toda la lógica de negocio del sistema. Es en esta capa donde se verifican las reglas del dominio (¿el email está duplicado? ¿la reserva se solapa

con otra? ¿el usuario tiene permiso para esta operación?), donde se orquestan las llamadas a múltiples repositorios si la operación así lo requiere, y donde se aplican las transformaciones de datos (cifrado de contraseñas, cálculo de franjas horarias, etc.). Los servicios son los únicos componentes que conocen las reglas de negocio; ni el controlador ni el repositorio deben contener lógica de este tipo.

- **4. Capa de repositorio:** las interfaces de repositorio, que extienden CrudRepository de Spring Data JPA, abstraen por completo el acceso a la base de datos. El servicio invoca métodos como save(), findById(), findAll() o consultas derivadas personalizadas (findByEmail, existsByNombreIgnoreCase), y Spring Data genera en tiempo de ejecución la implementación que traduce estas operaciones en las sentencias SQL correspondientes. El repositorio devuelve entidades JPA, que son objetos Java anotados que representan filas de las tablas de la base de datos.
- **5. Capa de entidades y base de datos:** las clases anotadas con @Entity (Usuario, Pista, Reserva, Disponibilidad, Rol) representan el modelo de datos persistente y son el objeto que circula entre las capas. Hibernate, el proveedor de JPA empleado por Spring Boot, gestiona automáticamente el ciclo de vida de estas entidades, rastreando sus cambios y sincronizándolos con la base de datos H2 cuando el contexto transaccional se confirma.

La respuesta recorre el camino inverso: el repositorio devuelve las entidades al servicio; el servicio aplica cualquier transformación necesaria (por ejemplo, convertir una entidad Usuario en un UsuarioResponse que no expone la contraseña) y devuelve el resultado al controlador; el controlador envuelve el resultado en un ResponseEntity con el código de estado HTTP apropiado; y Spring Boot serializa automáticamente el objeto a JSON y lo envía al cliente. Si en cualquier punto del flujo se produce un error de negocio, el servicio lanza una RuntimeException que el manejador global de errores intercepta y convierte en una respuesta HTTP con el código de estado adecuado.



3.1.2. Organización del código en paquetes

El código fuente del backend se distribuye en paquetes que reflejan fielmente la arquitectura de capas descrita. Bajo el paquete raíz `es.comillas.chispadel` se encuentran los siguientes subpaquetes:

- **controlador:** contiene la clase `ControladorREST`, que concentra todos los métodos manejadores de peticiones HTTP anotados con `@GetMapping`, `@PostMapping`, `@PatchMapping` y `@DeleteMapping`.
- **servicio:** agrupa las clases de servicio `UsuarioService`, `PistaService`, `ReservaService`, `DisponibilidadService` y `EmailService`, anotadas con `@Service` para que Spring las gestione como beans singletons.
- **repositorio:** reúne las interfaces de repositorio `RepoUsuario`, `RepoPista`, `RepoReserva`, `RepoRol` y `RepoDisponibilidad`, que extienden `CrudRepository` y, opcionalmente, añaden consultas derivadas personalizadas.
- **entity:** contiene las clases de entidad JPA `Usuario`, `Pista`, `Reserva`, `Disponibilidad` y `Rol`, anotadas con `@Entity` y mapeadas a las tablas de la base de datos.
- **model:** agrupa todo aquello que no encaja en las capas anteriores: los enumerados (`NombreRol`, `EstadoReserva`), los objetos de transferencia de datos o DTOs (`RegisterRequest`, `LoginRequest`, `LoginResponse`, `UsuarioResponse`, `DisponibilidadResponse`), las clases de configuración (`ConfigSeguridad`, `TestSecurityConfig`), las excepciones personalizadas (`ExcepcionUsuarioIncorrecto`) y los filtros (`BearerTokenFilter`).
- **programadas:** contiene la clase `TareasProgramadas`, que agrupa los métodos anotados con `@Scheduled` que se ejecutan automáticamente en momentos predefinidos.

Esta organización permite localizar con rapidez cualquier componente siguiendo una convención clara: si se busca la lógica de negocio de usuarios, se acude a `servicio.UsuarioService`; si se quiere inspeccionar la estructura de la entidad `Reserva`, se consulta `entity.Reserva`; si se necesita modificar el filtro de tokens, se edita `model.BearerTokenFilter`. La coherencia de esta estructura facilita la incorporación de nuevos miembros al equipo y reduce el tiempo necesario para comprender el código existente.

3.2. Modelo de datos y entidades JPA

El modelo de datos de ChisPadel se compone de cinco entidades principales que representan los conceptos del dominio del problema y que se mapean, mediante anotaciones de JPA, a tablas de la base de datos relacional H2. Además, dentro de las entidades se priorizará el uso de getters y setters, así como el uso de constructores.

3.2.1. Entidad Usuario

La entidad `Usuario` representa a cualquier persona que utiliza el sistema, ya sea un usuario estándar o un administrador. Se mapea a la tabla `USUARIO` mediante la anotación `@Entity` y define los siguientes campos, cada uno mapeado a una columna mediante `@Column`:

Campo	Tipo	Restricciones	Descripción
id	Long	@Id @GeneratedValue	Clave primaria generada automáticamente por la base de datos.

Campo	Tipo	Restricciones	Descripción
nombre	String	@Column(nullable=false)	Nombre de pila del usuario. Campo obligatorio.
apellidos	String	@Column	Apellidos del usuario. Campo opcional.
email	String	@Column(nullable=false, unique=true) @Email	Correo electrónico, único en el sistema y con validación de formato.
password	String	@Column(nullable=false)	Hash BCrypt de la contraseña. Nunca se almacena en texto plano.
telefono	String	@Column(nullable=false)	Número de teléfono del usuario.
rol	NombreRol	@Enumerated(String) @Column(nullable=false)	Rol del usuario: USER o ADMIN, almacenado como cadena.
fechaRegistro	LocalDateTime	@Column(nullable=false)	Fecha y hora de alta en el sistema.
activo	Boolean	@Column(nullable=false)	Indica si el usuario está habilitado.

El campo rol se anota con `@Enumerated(EnumType.STRING)`, lo que instruye a JPA para que almacene el valor del enumerado como su representación textual ("USER" o "ADMIN") en lugar de como su índice ordinal (0 o 1). Esta decisión tiene dos ventajas: en primer lugar, hace el esquema de base de datos legible directamente, de modo que al inspeccionar la tabla USUARIO se ve inmediatamente qué rol tiene cada usuario sin necesidad de recordar la correspondencia entre índices y valores; en segundo lugar, protege el esquema frente a reordenaciones del enumerado, ya que añadir un nuevo rol en medio de los existentes no alteraría los valores ya almacenados.

El campo password no se incluye en el método `toString()` para evitar su exposición accidental en logs.

3.2.2. Entidad Pista

La entidad Pista representa una instalación deportiva disponible para reserva, mapeada a la tabla PISTA. Sus campos son:

Campo	Tipo	Restricciones	Descripción
idPista	Long	@Id @GeneratedValue	Identificador único de la pista.
nombre	String	@Column(nullable=false, unique=true)	Nombre identificativo de la pista (ej: "Pista Central 1"). Debe ser único.
ubicacion	String	@Column(nullable=false)	Descripción de la ubicación física de la pista.
precioHora	Integer	@Column(nullable=false)	Precio de alquiler de la pista por hora, en céntimos de euro.
activa	Boolean	@Column(nullable=false)	Indica si la pista admite nuevas reservas.
fechaAlta	String	@Column(nullable=false)	Fecha de creación de la pista en formato ISO 8601.

El campo nombre está sujeto a una restricción de unicidad que la base de datos impone mediante un índice único. Cualquier intento de insertar o actualizar una pista con un nombre ya existente provoca una `DataIntegrityViolationException` que el servicio captura y convierte en un código de estado 409 Conflict. El campo activa controla la disponibilidad de la pista: una pista inactiva no puede recibir nuevas reservas, aunque las reservas existentes se mantienen. Esta separación

entre desactivación lógica y eliminación física permite preservar el histórico de reservas incluso cuando una pista deja de estar operativa.

3.2.3. Entidad Reserva

La entidad Reserva modela la reserva de una pista por parte de un usuario en una franja horaria concreta. Sus campos son:

Campo	Tipo	Restricciones	Descripción
reservationId	Long	@Id @GeneratedValue	Identificador único de la reserva.
usuario	Usuario	@ManyToOne(optional=false)	Usuario que realizó la reserva. Relación obligatoria.
pista	Pista	@ManyToOne(optional=false)	Pista reservada. Relación obligatoria.
inicio	Instant	@Column(nullable=false)	Instante de comienzo de la reserva en UTC.
fin	Instant	@Column(nullable=false)	Instante de finalización de la reserva en UTC.
fechaReservada	LocalDate	@Column(nullable=false)	Fecha de la reserva (sin componente horaria).
duracionMinutos	Integer	@Column	Duración en minutos. Calculado automáticamente si no se proporciona.
estado	EstadoReserva	@Enumerated(String) @Column(nullable=false)	Estado: ACTIVA o CANCELADA.
fechaCreacion	Instant	@Column(nullable=false)	Instante en que se creó la reserva.

Las relaciones @ManyToOne con Usuario y Pista son obligatorias (optional=false), lo que significa que no pueden existir reservas sin usuario o sin pista asociados. JPA materializa estas relaciones mediante claves ajenas en la tabla RESERVA que referencian las claves primarias de USUARIO y PISTA respectivamente. El uso de Instant en lugar de LocalDateTime para los campos inicio y fin garantiza que las reservas se almacenan en UTC, evitando problemas de zonas horarias y cambios de horario de verano.

El campo estado modela el ciclo de vida de una reserva. Cuando se crea, su estado es ACTIVA; cuando el usuario o un administrador la cancela, pasa a CANCELADA. La cancelación no elimina el registro de la base de datos, sino que actualiza este campo, lo que permite mantener un histórico completo de todas las reservas realizadas, algo valioso para auditorías y estadísticas. El campo duracionMinutos, aunque derivable restando fin e inicio, se persiste de forma redundante para facilitar consultas y ordenaciones por duración.

3.2.4. Entidad Disponibilidad

La entidad Disponibilidad recoge, para una pista y una fecha determinadas, el horario de apertura y cierre del club y la lista de franjas horarias libres, es decir, aquellos intervalos temporales en los que la pista no tiene ninguna reserva. Su estructura es:

Campo	Tipo	Descripción
disponibilidadId	Long	Identificador único de la disponibilidad.

Campo	Tipo	Descripción
pista	Pista	Pista a la que se refiere esta disponibilidad (relación @ManyToOne).
fecha	LocalDate	Fecha para la que se calcula la disponibilidad.
apertura	LocalTime	Hora de apertura del club ese día.
cierre	LocalTime	Hora de cierre del club ese día.
franjasLibres	List<FranjaDisponible>	Lista de franjas horarias libres (relación @OneToMany).

Cada franja libre se modela mediante la clase anidada `FranjaDisponible`, que es a su vez una entidad JPA con su propia tabla. `FranjaDisponible` contiene los campos `id`, `horaInicio`, `horaFin` y una referencia a la `Disponibilidad` a la que pertenece. La relación entre `Disponibilidad` y `FranjaDisponible` es `@OneToMany` bidireccional, lo que permite navegar desde una disponibilidad a todas sus franjas y viceversa.

La decisión de persistir la disponibilidad en lugar de calcularla siempre bajo demanda responde a un compromiso entre rendimiento y simplicidad. El cálculo de disponibilidad exige consultar todas las reservas de la pista para el día en cuestión, ordenarlas por hora de inicio, detectar los huecos entre reservas y comparar estos huecos con el horario de apertura y cierre. Esta operación, aunque no especialmente costosa, se ejecuta frecuentemente cuando los usuarios consultan las pistas disponibles, por lo que pre-calcularla y almacenarla reduce la carga de la base de datos. El inconveniente es la necesidad de mantener sincronizada la disponibilidad con las reservas: cada vez que se crea, modifica o cancela una reserva, se debe recalcular la disponibilidad afectada.

3.2.5. Entidad Rol y enumerados

La entidad `Rol` permite definir roles del sistema de forma independiente de la entidad `Usuario`, con su identificador, su nombre y una descripción explicativa. Aunque en la implementación actual del proyecto el rol se almacena directamente en `Usuario` como un campo enumerado, la entidad `Rol` se ha incluido por completitud del modelo, permitiendo en el futuro asociar permisos más granulares a cada rol sin modificar la estructura de `Usuario`.

El sistema emplea dos *enums* principales:

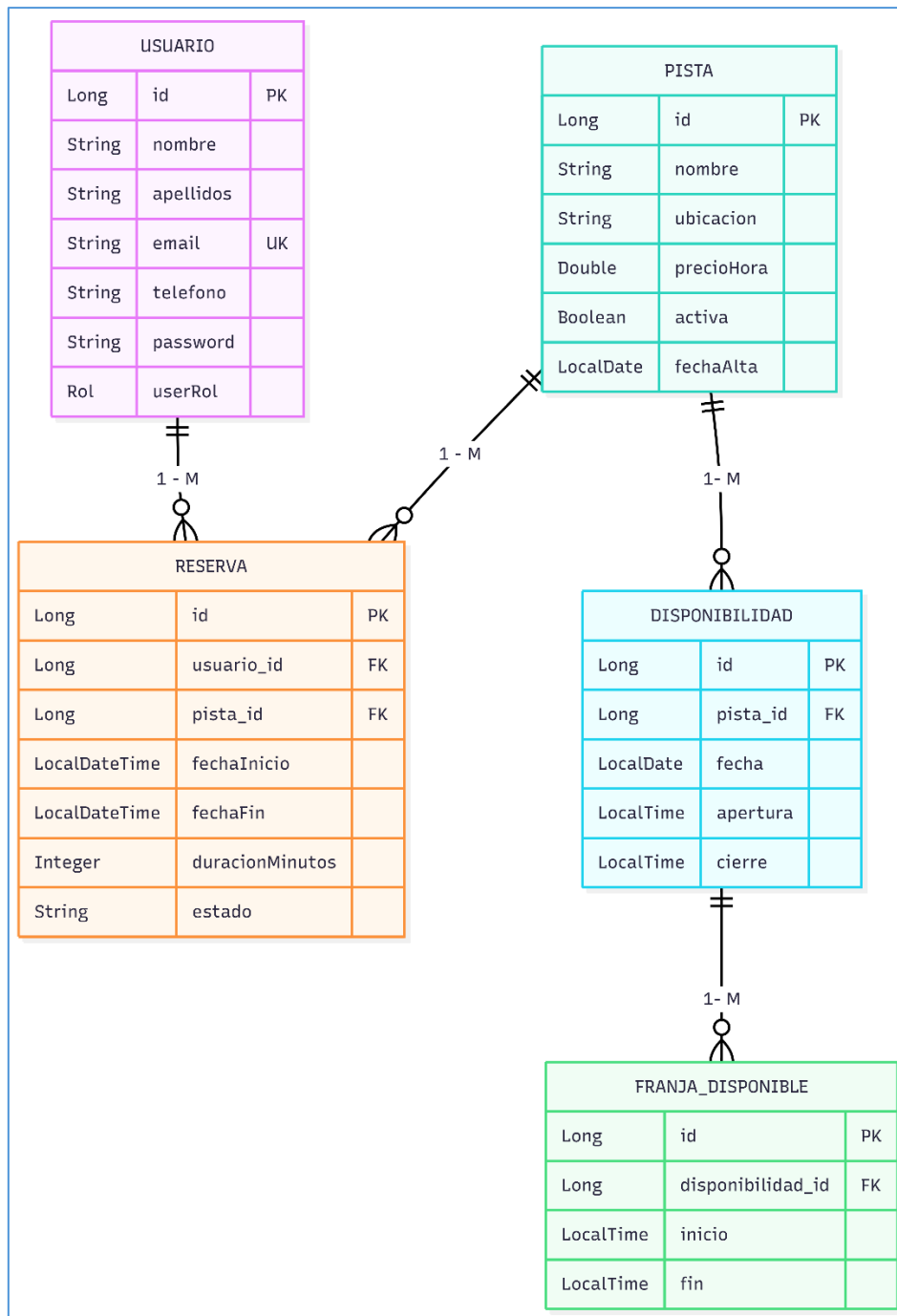
- **NombreRol:** define los dos roles posibles del sistema, `USER` y `ADMIN`. Se utiliza como tipo del campo `rol` en `Usuario` y se referencia en múltiples comprobaciones de autorización.
- **EstadoReserva:** define los dos estados posibles de una reserva, `ACTIVA` y `CANCELADA`. Se utiliza como tipo del campo `estado` en `Reserva` y permite filtrar reservas por su estado.

3.2.6. Relaciones entre entidades y diagrama ER

Las entidades se relacionan entre sí conforme al siguiente esquema, que refleja las cardinalidades del dominio del problema:

- Un `Usuario` puede tener de cero a muchas `Reservas`, y cada `Reserva` pertenece a exactamente un `Usuario`. Relación 1:M materializada mediante clave ajena en la tabla `RESERVA`.
- Una `Pista` puede tener de cero a muchas `Reservas`, y cada `Reserva` se asocia a exactamente una `Pista`. Relación 1:M materializada mediante clave ajena en la tabla `RESERVA`.

- Una Pista puede tener de cero a muchas Disponibilidades (una por fecha distinta), y cada Disponibilidad se refiere a exactamente una Pista. Relación 1:M.
- Una Disponibilidad puede tener de cero a muchas FranjasDisponibles, y cada FranjaDisponible pertenece a exactamente una Disponibilidad. Relación 1:M con cascada de operaciones.
- Un Usuario tiene asignado exactamente un Rol. Aunque modelado como relación en la entidad Rol independiente, en la práctica se simplifica como campo enumerado en Usuario.



3.3. Capa de persistencia: los repositorios

El acceso a la base de datos se canaliza a través de interfaces de repositorio que extienden `CrudRepository` de Spring Data JPA. Esta tecnología, ampliamente adoptada en el ecosistema Spring, proporciona automáticamente las operaciones CRUD básicas —`save()`, `findById()`, `findAll()`, `delete()`— sin necesidad de escribir una sola línea de código de implementación. Spring Data genera en tiempo de ejecución una clase proxy que implementa la interfaz, traduciendo las invocaciones de métodos en las consultas SQL correspondientes mediante Hibernate.

Además de las operaciones estándar, Spring Data JPA permite definir consultas personalizadas mediante dos mecanismos complementarios: las consultas derivadas del nombre del método y las consultas JPQL anotadas. Las consultas derivadas explotan una convención de nombres muy expresiva: si se declara un método `findByEmail(String email)` en la interfaz, Spring Data interpreta que debe generar una consulta `SELECT WHERE email = ?`. Este mecanismo cubre la mayoría de los casos de uso habituales —búsquedas por igualdad, por rango, por ordenación— sin necesidad de escribir SQL explícito.

3.3.1. RepoUsuario

El repositorio de usuarios, extiende `CrudRepository<Usuario, Long>` e incorpora tres consultas:

```
public interface RepoUsuario extends CrudRepository<Usuario, Long> {
    Usuario findByEmail(String email);
    boolean existsByEmail(String email);
    boolean existsByEmailAndIdNot(String email, Long id);
}
```

- **findByEmail:** recupera un usuario a partir de su correo electrónico. Es la consulta fundamental del proceso de login: el servicio busca al usuario por su email, verifica que existe y está activo, y compara la contraseña hashada almacenada con la proporcionada por el cliente.
- **existsByEmail:** comprueba la existencia de un usuario con un correo determinado sin necesidad de recuperar el objeto completo. Se utiliza en el registro para detectar duplicados antes de persistir el nuevo usuario: si ya existe alguien con ese email, se rechaza el registro con un 409 Conflict. Esta consulta es más eficiente que `findByEmail` cuando solo interesa saber si existe, porque la base de datos puede responder sin materializar la fila completa.
- **existsByEmailAndIdNot:** verifica que un correo no pertenece a otro usuario distinto del indicado por el parámetro `id`. Se emplea en la edición de perfil para validar que el nuevo email no está en uso, pero sin rechazar el propio email del usuario que está editando. La lógica es: "¿existe algún usuario con este email cuyo identificador no sea el mío?". Si la respuesta es no, el email está disponible; si es sí, pertenece a otra persona y no puede usarse.

3.3.2. RepoPista

El repositorio de pistas, `RepoPista`, añade a las operaciones básicas dos consultas:

```
public interface RepoPista extends CrudRepository<Pista, Long> {
    boolean existsByNombreIgnoreCase(String nombre);
    boolean existsByNombreIgnoreCaseAndIdPistaNot(String nombre, Long id);
}
```

Ambas consultas verifican la unicidad del nombre de las pistas, ignorando mayúsculas y minúsculas mediante el modificador `IgnoreCase`. La primera se usa al crear una pista nueva; la segunda, al modificarla, para no rechazar el propio nombre de la pista que se está editando.

3.3.3. RepoReserva

El repositorio de reservas, `RepoReserva`, es el más complejo de todos debido a la necesidad de detectar solapamientos temporales.

```
List<Reserva> findByPistaAndInicioBeforeAndFinAfterAndEstado(  
    Pista pista, Instant fin, Instant inicio, EstadoReserva estado);
```

Esta consulta devuelve todas las reservas activas de una pista cuyo intervalo [inicio, fin) se solapa con el intervalo solicitado. Dos intervalos se solapan si y solo si el inicio de uno es anterior al fin del otro y viceversa. La consulta se invoca tanto al crear una reserva como al reprogramarla; si devuelve algún resultado, existe conflicto y la operación se rechaza con un 409.

Otras consultas derivadas de este repositorio permiten filtrar reservas por usuario, por pista, por rango de fechas o por combinaciones de estos criterios, proporcionando la flexibilidad necesaria para los distintos endpoints de consulta.

3.3.4. RepoDisponibilidad y RepoRol

El repositorio de disponibilidad incorpora métodos para localizar la disponibilidad de una pista en una fecha concreta y para listar todas las disponibilidades de una fecha determinada, filtrando opcionalmente por pista. Mientras que, `RepoRol`, proporciona únicamente las operaciones CRUD estándar.

3.4. Objetos de transferencia de datos (DTOs)

Uno de los principios del diseño de APIs REST es no exponer directamente las entidades de dominio en las respuestas, sino emplear objetos de transferencia de datos (DTOs) que contengan únicamente la información que el cliente necesita conocer. Esta separación aporta múltiples beneficios: protege información sensible, desacopla el contrato de la API de la estructura interna de las entidades, permite versionar la API sin modificar el modelo de datos y facilita la evolución independiente del frontend y del backend.

ChisPadel emplea DTOs tanto para las peticiones (request objects) como para las respuestas (response objects). A continuación se describen los principales:

3.4.1. RegisterRequest

DTO que recoge los campos necesarios para registrar un nuevo usuario. Se declara como clase con los siguientes campos, validados mediante anotaciones de Bean Validation:

```
@NotBlank(message = "El nombre es obligatorio")  
private String nombre;  
  
private String apellidos; // Opcional
```



```

    @Email(message = "El email debe tener un formato válido")
    @NotBlank(message = "El email es obligatorio")
    private String email;

    @Size(min = 6, message = "La contraseña debe tener al menos 6 caracteres")
    @NotBlank(message = "La contraseña es obligatoria")
    private String password;

    @NotBlank(message = "El teléfono es obligatorio")
    private String telefono;

```

El controlador recibe este DTO anotado con `@Valid`, de modo que Bean Validation rechaza automáticamente la petición con un 400 Bad Request si alguna restricción no se cumple, antes de que el método del controlador se ejecute.

3.4.2. LoginRequest y LoginResponse

El DTO de petición de login se implementa como un record de Java, una característica introducida en Java 14 que simplifica la declaración de clases inmutables de datos. Un record genera automáticamente el constructor, los getters, equals, hashCode y toString:

```

public record LoginRequest(
    @Email @NotBlank String email,
    @NotBlank String password
) {}

```

LoginResponse contiene únicamente el token UUID generado por el servidor tras una autenticación exitosa. Este diseño minimalista sigue el principio de no exponer más información de la necesaria: el cliente obtiene el token y, si necesita conocer datos adicionales del usuario, realiza una segunda petición a `/auth/me`.

3.4.3. UsuarioResponse

DTO de respuesta que representa un usuario de forma pública. Contiene todos los campos informativos —id, nombre, apellidos, email, teléfono, rol, fechaRegistro, activo— pero excluye intencionadamente el campo password, evitando que el hash BCrypt sea expuesto en las respuestas de la API. Este DTO se emplea en los endpoints `/auth/me`, `/users` y `/users/{userId}`.

3.4.4. DisponibilidadResponse

DTO que modela la disponibilidad de una pista para una fecha. Contiene el identificador y nombre de la pista, la fecha, las horas de apertura y cierre, y la lista de franjas libres. Cada franja libre se representa a su vez mediante un DTO anidado con los campos `horaInicio` y `horaFin`. Este objeto se construye a partir de las entidades Disponibilidad y FranjaDisponible.

3.5. Subsistema de seguridad

La seguridad es uno de los aspectos más críticos del proyecto. El sistema debe garantizar que solo los usuarios autenticados accedan a los recursos protegidos, que cada usuario únicamente pueda realizar las operaciones acordes a su rol y que las contraseñas se almacenen de forma segura. Para ello se ha empleado Spring Security, el módulo de seguridad de referencia del ecosistema Spring, complementado con un filtro personalizado de autenticación por tokens y con una configuración específica para el entorno de pruebas.

3.5.1. Configuración principal: ConfigSeguridad

La clase ConfigSeguridad, anotada con `@EnableWebSecurity` y `@EnableMethodSecurity`, concentra la configuración de seguridad del entorno de producción. Se activa únicamente fuera del perfil de pruebas mediante `@Profile("!test")`. Su método principal, `filterChain`, define un `SecurityFilterChain` que configura los siguientes aspectos:

- **CORS (Cross-Origin Resource Sharing):** se configura un `CorsConfigurationSource` que permite todas las peticiones desde cualquier origen (`allowedOriginPatterns("*")`), todos los métodos HTTP relevantes (GET, POST, PATCH, DELETE, OPTIONS) y todas las cabeceras, con credenciales habilitadas (`allowCredentials(true)`). Esta configuración permisiva es necesaria en desarrollo para que el frontend, servido por Live Server en un puerto distinto, pueda comunicarse con el backend en `localhost:8080`. En producción, `allowedOriginPatterns` debería restringirse a los orígenes concretos autorizados.
- **CSRF (Cross-Site Request Forgery):** se deshabilita mediante `csrf().disable()`, ya que la API REST es sin estado y se basa en tokens Bearer incluidos en las cabeceras HTTP en lugar de en cookies de sesión. CSRF protege contra ataques que explotan cookies automáticas del navegador, mecanismo que la API no emplea.
- **Frame options:** se desactivan con `frameOptions().disable()` para permitir que la consola web de la base de datos H2 se muestre dentro de un `iframe`. En producción, donde H2 no estaría expuesto, esta opción debería mantenerse activa.
- **Autorización de rutas:** los endpoints de autenticación (`/pistaPadel/auth/**`), usuarios (`/pistaPadel/users/**`), pistas (`/pistaPadel/courts/**`), reservas (`/pistaPadel/reservations/**`), disponibilidad (`/pistaPadel/availability/**`) y consulta de administración (`/pistaPadel/admin/**`) se declaran accesibles mediante `permitAll()`, de modo que el filtro de seguridad no los bloquee antes de que la petición alcance el controlador. Esta decisión traslada la responsabilidad de la autorización fina al servicio y a las anotaciones `@PreAuthorize` del controlador, permitiendo un control más granular y expresivo. El endpoint de `healthcheck` y la consola H2 también se permiten sin autenticación. El resto de rutas requieren autenticación.
- **Filtro de token Bearer:** se registra antes de `UsernamePasswordAuthenticationFilter` mediante `addFilterBefore()`, de forma que el token Bearer del frontend se procesa y el `SecurityContextHolder` se puebla con la autoridad del usuario antes de que Spring Security evalúe cualquier anotación de control de acceso.
- **HTTP Basic:** se habilita con la configuración por defecto mediante `httpBasic(Customizer.withDefaults())`, manteniendo compatibilidad con los tests E2E que emplean credenciales básicas en lugar de tokens. En producción, HTTP Basic debería deshabilitarse.

El diseño de la seguridad distingue dos niveles de control claramente diferenciados. El primer nivel es el control de autenticación, que comprueba si quien realiza la petición ha iniciado sesión; su ausencia produce una respuesta 401 Unauthorized. El segundo nivel es el control de autorización, que comprueba si el usuario autenticado tiene permiso para la operación concreta solicitada; su incumplimiento produce una respuesta 403 Forbidden. Esta distinción entre el 401 y el 403 se ha respetado de forma rigurosa en toda la API, lo que facilita enormemente la depuración de problemas de permisos y mejora la experiencia del usuario al proporcionar mensajes de error precisos.

The first screenshot shows a GET request to `http://localhost:8080/pistaPadel/auth/me`. The status is **401 Unauthorized**. The response body is empty.

The second screenshot shows a GET request to `http://localhost:8080/pistaPadel/users`. The status is **403 Forbidden**. The response body is a JSON object:

```

{
  "timestamp": "2026-05-17T19:46:39.576+00:00",
  "status": 403,
  "error": "Forbidden",
  "path": "/pistaPadel/users"
}

```

3.5.2. El filtro BearerTokenFilter

La autenticación de ChisPadel se basa en tokens de tipo Bearer. Cuando un usuario inicia sesión exitosamente, el servidor genera un token —un identificador universal único (UUID)— y lo entrega al cliente en la respuesta. En las peticiones posteriores, el cliente debe incluir dicho token en la cabecera HTTP Authorization con el formato "Bearer <token>". La clase `BearerTokenFilter`, que extiende `OncePerRequestFilter` —garantizando así que se ejecuta una única vez por petición—, se encarga de procesar este token en cada petición entrante.

El filtro implementa la siguiente lógica en su método `doFilterInternal`:

1. Extrae la cabecera `Authorization` de la petición mediante `UsuarioService.extractToken()`, que parsea la cabecera y devuelve únicamente el UUID sin el prefijo "Bearer".
2. Si el token está presente, lo verifica contra el almacén de tokens llamando a `usuarioService.verifyAndGetUserId(token)`. Este método comprueba que el token existe en el mapa, que no ha caducado y, si todo es correcto, devuelve el `userId` asociado.
3. Recupera el usuario completo de la base de datos mediante `usuarioService.getUsuarioById(userId)`.
4. Construye un objeto `UsernamePasswordAuthenticationToken`, que es la clase de autenticación estándar de Spring Security, con el usuario como principal, null como credenciales (ya no se necesitan, el usuario ya ha sido verificado) y una colección con una única autoridad: `ROLE_ADMIN` o `ROLE_USER` según el rol del usuario. El prefijo "ROLE_" es requerido por Spring Security para las expresiones `@PreAuthorize("hasRole(...)")`.

5. Deposita este objeto de autenticación en el SecurityContextHolder, que es un almacén estático thread-local donde Spring Security guarda el contexto de seguridad de la petición actual.
6. Invoca filterChain.doFilter() para que la petición continúe su camino hacia el siguiente filtro y finalmente hacia el controlador.

El filtro solo actúa si el SecurityContextHolder está vacío (para no sobrescribir autenticaciones ya establecidas por otros mecanismos, como HTTP Basic en los tests) y omite las rutas de la consola H2 mediante una verificación en shouldNotFilter(). Gracias a este filtro, cuando la petición llega al controlador, el SecurityContextHolder ya contiene la autenticación poblada con las autoridades correctas, de modo que las anotaciones @PreAuthorize("hasRole('ADMIN')") funcionan correctamente.

```
@Override
protected void doFilterInternal(HttpServletRequest request,
                                HttpServletResponse response,
                                FilterChain filterChain)
    throws ServletException, IOException {
    if (SecurityContextHolder.getContext().getAuthentication() != null) {
        filterChain.doFilter(request, response);
        return;
    }
    String token = usuarioService.extractToken(
        request.getHeader("Authorization"));
    if (token != null) {
        try {
            Long userId = usuarioService.verifyAndGetUserId(token);
            Usuario usuario = usuarioService.getUsuarioById(userId);
            Authentication auth = new UsernamePasswordAuthenticationToken(
                usuario, null,
                List.of(new SimpleGrantedAuthority(
                    "ROLE_" + usuario.getRol().name())));
            SecurityContextHolder.getContext().setAuthentication(auth);
        } catch (Exception e) {
            // Token inválido o caducado, se deja pasar sin autenticación
        }
    }
    filterChain.doFilter(request, response);
}
```

3.5.3. Gestión de tokens en UsuarioService

El almacén de tokens se implementa en UsuarioService como un ConcurrentHashMap<String, TokenInfo>, donde la clave es el UUID del token y el valor es un objeto TokenInfo que contiene el userId y un Instant de expiración fijado a ocho horas desde la emisión. Esta estructura en memoria es simple y suficiente para un proyecto académico, aunque en un sistema de producción los tokens deberían persistirse en una base de datos o en un caché distribuido como Redis para soportar escalado horizontal.

Los métodos que gestionan el ciclo de vida de los tokens son:

- **issueToken(Usuario u):** genera un UUID aleatorio, calcula su instante de expiración sumando 8 horas al instante actual, inserta la entrada en el mapa y devuelve el UUID. Se invoca tras un login exitoso.
- **verifyAndGetUserId(String token):** busca el token en el mapa, comprueba que no ha caducado comparando su instante de expiración con Instant.now(), y devuelve el userId. Si el token no existe o ha caducado, elimina la entrada del mapa y lanza una excepción. Se invoca desde el filtro y desde los endpoints que requieren autenticación.

- **extractToken(String authHeader):** método estático que parsea la cabecera Authorization, comprueba que empieza por "Bearer " (con espacio) y devuelve el substring restante, que es el token. Devuelve null si la cabecera es nula o no tiene el prefijo esperado.
- **invalidateToken(String token):** elimina la entrada del mapa, invalidando el token de forma inmediata. Se invoca al cerrar sesión.

3.5.4. Cifrado de contraseñas con BCrypt

Las contraseñas de los usuarios nunca se almacenan en texto plano. En el momento del registro, la contraseña proporcionada por el cliente se cifra mediante BCrypt, un algoritmo de hashing diseñado específicamente para contraseñas que incorpora una sal aleatoria y un factor de coste configurable que ralentiza deliberadamente el cálculo, dificultando los ataques de fuerza bruta.

Spring Security proporciona la clase BCryptPasswordEncoder, que el proyecto instancia como bean singleton en la configuración. Durante el registro, el servicio invoca `encoder.encode(password)` para obtener el hash, que se persiste en el campo `password` de Usuario. Durante el login, el servicio invoca `encoder.matches(passwordIntroducida, passwordAlmacenado)` para comprobar si la contraseña proporcionada por el cliente, al ser hasheada, produce el mismo resultado que el almacenado. Esta comparación se realiza sin necesidad de descifrar el hash, ya que BCrypt es una función unidireccional.

```
// Registro
String hashBCrypt = passwordEncoder.encode(request.password());
usuario.setPassword(hashBCrypt);

// Login
if (!passwordEncoder.matches(request.password(), usuario.getPassword())) {
    throw new ResponseStatusException(HttpStatus.UNAUTHORIZED,
        "Credenciales incorrectas");
}
```

3.5.5. Configuración para pruebas: TestSecurityConfig

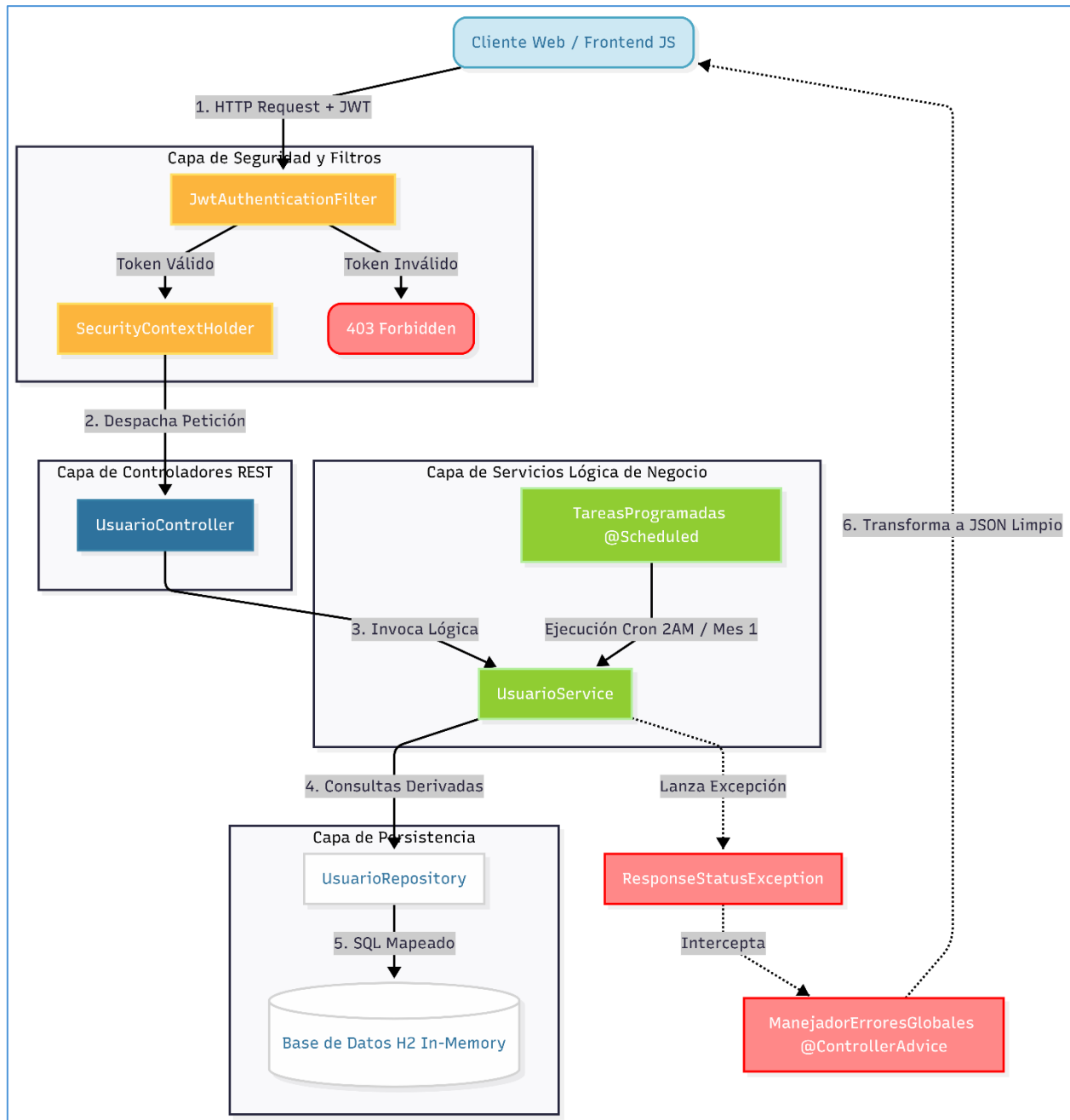
Para que las pruebas automatizadas puedan ejecutar los endpoints sin necesidad de proporcionar tokens en cada llamada, se ha definido `TestSecurityConfig`, anotada con `@Profile("test")`. Esta configuración alternativa define un `SecurityFilterChain` que:

- Permite todas las peticiones sin autenticación mediante `anyRequest().permitAll()`.
- Deshabilita CSRF, el formulario de login y HTTP Basic.
- Desactiva la evaluación de las anotaciones `@PreAuthorize` mediante `@EnableMethodSecurity(prePostEnabled = false)`.

De este modo, cuando los tests E2E se ejecutan con el perfil "test" activo, la capa de seguridad no interfiere, y las pruebas pueden centrarse en verificar la lógica de negocio sin ruido adicional. Esta separación entre la configuración de producción y la de pruebas, controlada mediante perfiles de Spring, ha sido una de las aportaciones más valiosas del proyecto, ya que permitió aislar correctamente el entorno de test sin contaminar el código de producción.

3.6. Implementación de los endpoints de la API

En esta sección se detallarán de forma exhaustiva los veintitres endpoints implementados, agrupados por el miembro del equipo responsable de cada bloque funcional. Para cada endpoint se describirá su propósito, su contrato (parámetros de entrada y estructura de respuesta), las validaciones que aplica, el código de estado HTTP que devuelve en cada situación y ejemplos de uso con capturas de Postman.



3.6.1. Endpoints del Bloque de Pistas

Endpoint 1: Registro de Nueva Pista

- **Ruta de Acceso:** POST /pistaPadel/courts

- **Seguridad y Permisos:** Autenticación requerida mediante JWT. Restringido estrictamente a usuarios que dispongan del rol `ROLE_ADMIN`.
- **Estructura del Cuerpo de la Petición (Request Body):**

```
{
  "nombre": "Madrid central 1",
  "ubicacion": "Madrid",
  "precioHora": 10.0,
  "activa": true,
  "fechaAlta": "2026-02-15"
}
```

- **Lógica Funcional y Validaciones:** El método intercepta el objeto DTO de la pista. Verifica que el campo nombre no coincida con ningún registro persistido en la base de datos de pistas. Si se detecta un duplicado, se aborta la transacción arrojando una excepción de negocio. Los valores numéricos de precio deben ser estrictamente positivos.
- **Códigos de Estado HTTP Devueltos:**
 - 201 Created: Operación exitosa. Retorna el objeto unificado incluyendo el ID autogenerated.
 - 400 Bad Request: Datos de entrada mal formados, cadena de texto vacía o tipos de datos erróneos.
 - 401 Unauthorized: Ausencia de la cabecera con el token de portador válido.
 - 403 Forbidden: Token válido pero perteneciente a un rol sin privilegios de administración (ej. `ROLE_USER`).
- **Ejemplo de Trazabilidad y Captura:**

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8080/pistaPadel/courts`
- Method:** `POST`
- Request Body (raw):**

```
{
  "nombre": "Pista Central",
  "ubicacion": "Madrid - Centro Deportivo Norte",
  "precioHora": 25,
  "activa": true
}
```
- Response:** `201 Created` (150 ms, 539 B)
- Response Body (JSON):**

```
{
  "id": 3,
  "nombre": "Pista Central",
  "ubicacion": "Madrid - Centro Deportivo Norte",
  "precioHora": 25,
  "activa": true,
  "fechaAlta": "2026-05-17"
}
```

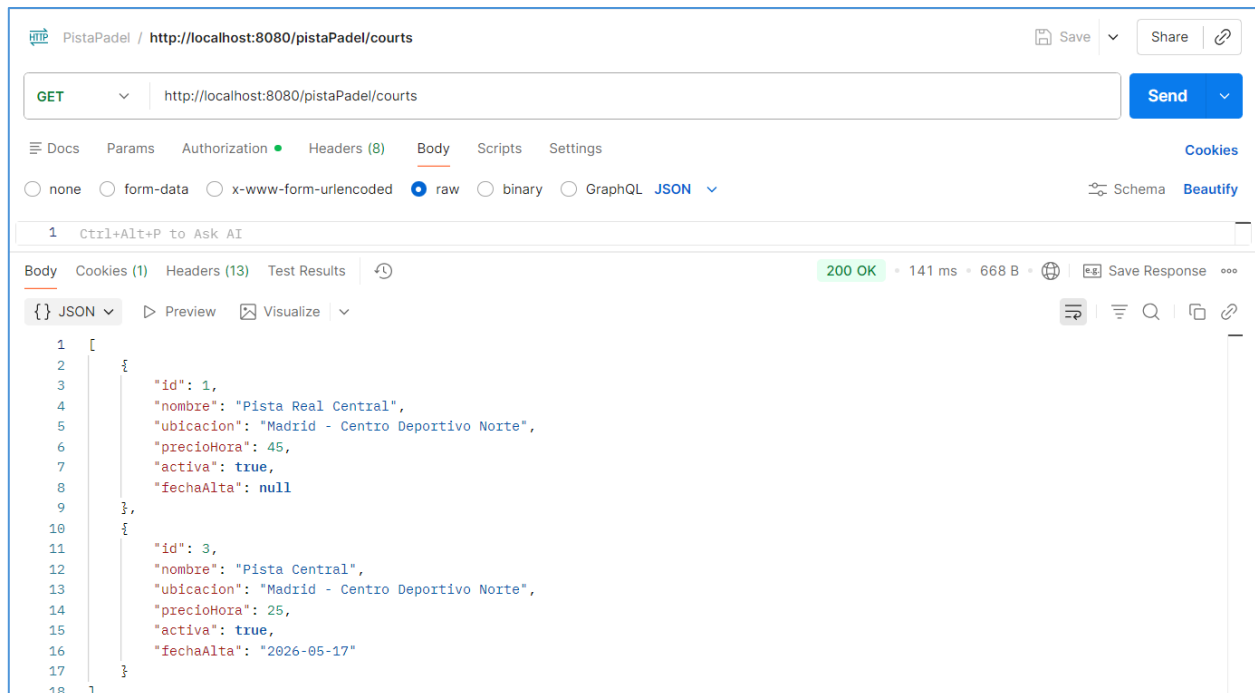
Endpoint 2: Recuperación General del Catálogo de Pistas

- **Ruta de Acceso:** `GET /pistaPadel/courts`

- **Seguridad y Permisos:** Endpoint público para usuarios autenticados con rol `ROLE_USER` o `ROLE_ADMIN`.
- **Estructura de la Respuesta (Response Body):**

```
{
  "id": 1,
  "nombre": "Madrid central 1",
  "ubicacion": "Madrid",
  "precioHora": 10.0,
  "activa": true,
  "fechaAlta": "2026-02-15T00:00:00"
}
```

- **Lógica Funcional:** Realiza una consulta total a la base de datos a través de la interfaz de persistencia de Spring Data. Si no se encuentran registros, devuelve una lista vacía con un código de éxito para no romper la renderización de la interfaz del cliente.
- **Códigos de Estado HTTP Devueltos:**
 - 200 OK: Petición atendida satisfactoriamente con la colección completa en el cuerpo.
 - 401 Unauthorized: Intento de acceso sin credenciales válidas.
- **Ejemplo de Trazabilidad y Captura:**



Endpoint 3: Modificación Parcial de Propiedades de una Pista

- **Ruta de Acceso:** `PATCH /pistaPadel/courts/{courtId}`
- **Seguridad y Permisos:** Requiere privilegios de nivel `ROLE_ADMIN`.
- **Estructura del Cuerpo de la Petición:**

```
{
  "precioHora": 12.5,
  "activa": false
}
```

- **Lógica Funcional:** Localiza la entidad por su clave primaria (id). Si no existe, se gestiona el error. Si se encuentra, el servicio aplica de manera selectiva las propiedades no nulas suministradas en el DTO, preservando el histórico y la fecha de alta original de la pista.
- **Códigos de Estado HTTP Devueltos:**
 - 200 OK: Modificación consolidada con éxito en la base de datos relacional.
 - 404 Not Found: El identificador de pista provisto en la ruta no corresponde a ningún registro real.

Endpoint 4: Eliminación del Registro de una Pista

- **Ruta de Acceso:** DELETE /pistaPadel/courts/{courtId}
- **Seguridad y Permisos:** Operación exclusiva para el rol ROLE_ADMIN.
- **Lógica Funcional:** Evalúa si la pista tiene dependencias activas o reservas vinculadas en el futuro. Si existen compromisos de juego vigentes, se deniega la eliminación física y se sugiere la desactivación lógica mediante el estado activa = false para preservar la integridad referencial de la base de datos.
- **Códigos de Estado HTTP Devueltos:**
 - 204 No Content: Eliminación completada con éxito. El cuerpo de la respuesta se mantiene vacío.
 - 400 Bad Request: Imposibilidad de borrado por restricciones de clave foránea activas.

3.6.2. Endpoints del Bloque de Disponibilidad

Endpoint 5: Consulta de Disponibilidad Consolidada por Fecha

- **Ruta de Acceso:** GET /pistaPadel/availability
- **Parámetros de Consulta (Query Params):** * date (Obligatorio, formato: YYYY-MM-DD)
 - courtId (Opcional, filtro numérico por pista)
- **Estructura de la Respuesta:**

```
{
  "pista": {
    "id": 1,
    "nombre": "Madrid central 1"
  },
  "fecha": "2026-06-10",
  "apertura": "08:00:00",
  "cierre": "23:00:00",
  "franjasLibres": [
    {
```

```

      "inicio": "09:30:00",
      "fin": "11:00:00"
    }
  ]
}

```

- **Lógica Funcional:** El servicio recibe la fecha de consulta y calcula el mapa de franjas horarias teóricas del club (ej. intervalos de 90 minutos de 08:00 a 23:00). Posteriormente, interroga al repositorio de reservas para obtener todas las reservas confirmadas de esa fecha. Mediante un algoritmo de exclusión de intervalos temporales (*Time-slot exclusion algorithm*), el servicio resta los bloques ocupados y compila las franjas vacantes resultantes.
- **Códigos de Estado HTTP Devueltos:**
 - 200 OK: Retorna el mapa detallado de disponibilidad de las pistas solicitadas.
 - 400 Bad Request: Parámetro date ausente, mal formateado o con una sintaxis temporal inválida.
- **Ejemplo de Trazabilidad y Captura:**

PistaPadel / <http://localhost:8080/pistaPadel/courts/2/availability>

GET <http://localhost:8080/pistaPadel/courts/1/availability?date=2025-06-01> Send

Docs Params Authorization Headers (7) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
date	2025-06-01		

Body Cookies (1) Headers (13) Test Results 200 OK • 40 ms • 675 B Save Response

```

1  {
2    "id": 1,
3    "pista": {
4      "id": 1,
5      "nombre": "Pista Real Central",
6      "ubicacion": "Madrid - Centro Deportivo Norte",
7      "precioHora": 45,
8      "activa": true,
9      "fechaAlta": null
10   },
11   "fecha": "2025-06-01",
12   "apertura": "08:00:00",
13   "cierre": "22:00:00",
14   "franjasLibres": [
15     {
16       "id": 1,
17       "inicio": "08:00:00",
18       "fin": "10:00:00"
19     }
20   ]
21 }

```

Endpoint 6: Inserción Manual de Disponibilidad / Bloqueos Técnicos

- **Ruta de Acceso:** POST /pistaPadel/availability/schedule
- **Seguridad y Permisos:** Acceso exclusivo para el rol ROLE_ADMIN.
- **Estructura del Cuerpo de la Petición:**

```

{
  "pistaId": 1,
  "fecha": "2026-06-11",

```

```
"motivo": "Mantenimiento Técnico - Sustitución de Red"
}
```

- **Lógica Funcional:** Permite al administrador reservar de manera forzada jornadas completas de una pista para labores de mantenimiento o eventos corporativos, anulando la generación automática de franjas libres para los usuarios comunes.
- **Códigos de Estado HTTP Devueltos:**
 - 201 Created: Bloqueo persistido correctamente en la base de datos de disponibilidad.

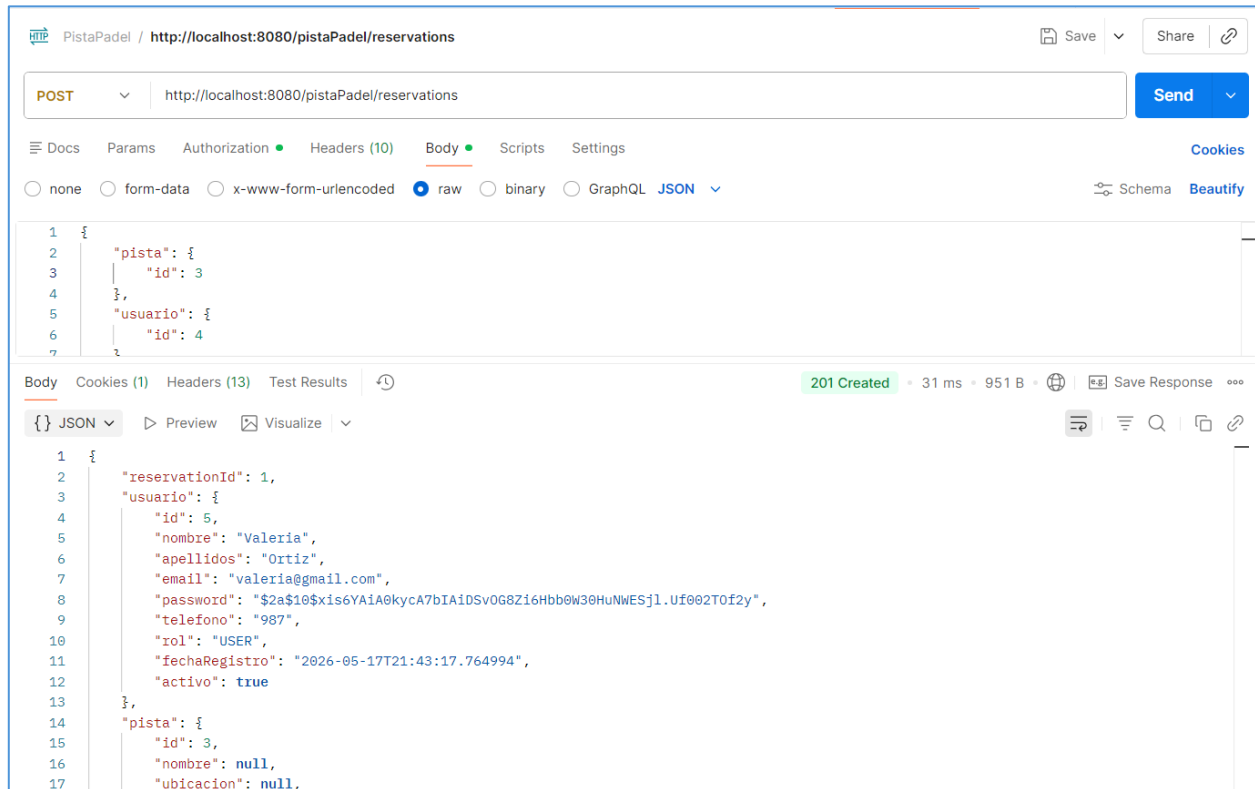
3.6.3. Endpoints del Bloque de Reservas

Endpoint 7: Creación y Formalización de Reserva

- **Ruta de Acceso:** POST /pistaPadel/reservations
- **Seguridad y Permisos:** Requiere autenticación de usuario (ROLE_USER o ROLE_ADMIN).
- **Estructura del Cuerpo de la Petición:**

```
{
  "pistaId": 1,
  "fechaInicio": "2026-06-10T18:00:00",
  "duracionMinutos": 90
}
```

- **Lógica Funcional:** El controlador extrae la identidad del usuario a partir del token JWT inyectado en el contexto de seguridad de Spring Security. El servicio valida que la pista se encuentre activa y que el intervalo de tiempo solicitado (fechaInicio hasta fechaInicio + duracionMinutos) esté completamente libre en el módulo de disponibilidad. Se aplican bloqueos optimistas para impedir que dos peticiones simultáneas reserven la misma pista en el mismo intervalo (*Race Condition*).
- **Códigos de Estado HTTP Devueltos:**
 - 201 Created: Reserva confirmada. Se devuelve el objeto de reserva completo con los detalles del usuario y de la pista asociada.
 - 400 Bad Request: Conflicto de solapamiento horario con una reserva preexistente o duración no permitida por la normativa del club.
- **Ejemplo de Trazabilidad y Captura:**



Endpoint 8: Obtención de Reservas del Usuario Autenticado

- **Ruta de Acceso:** GET `/pistaPadel/reservations/{reservationId}`
- **Seguridad y Permisos:** Requiere token válido. Retorna exclusivamente los datos del firmante del token.
- **Estructura de la Respuesta:**

```

{
  "id": 102,
  "fechaInicio": "2026-06-10T18:00:00",
  "duracionMinutos": 90,
  "pista": {
    "id": 1,
    "nombre": "Madrid central 1"
  }
}

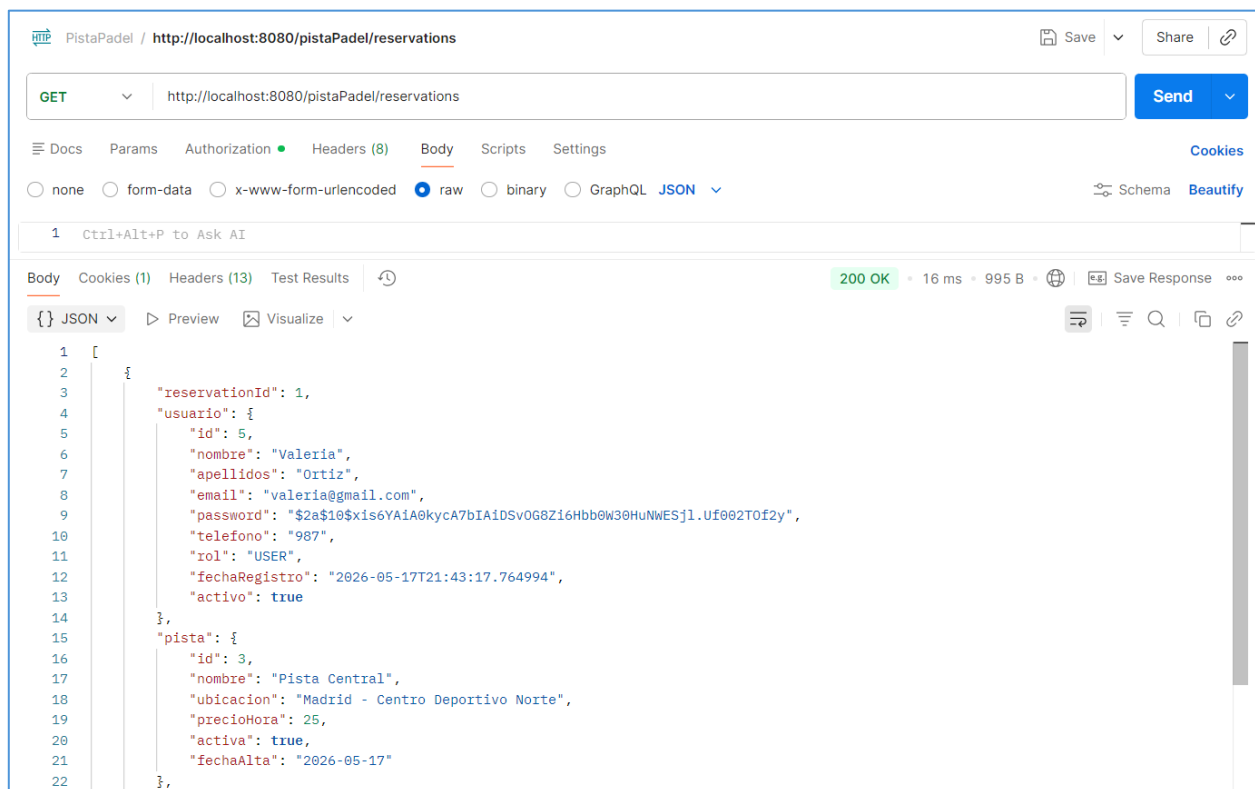
```

- **Lógica Funcional:** Recupera el identificador de usuario desde el contexto de seguridad y delega en el repositorio la ejecución de una consulta filtrada por la clave foránea del usuario.
- **Códigos de Estado HTTP Devueltos:**
 - 200 OK: Retorna el listado histórico de reservas del cliente actual de la sesión.

Endpoint 9: Consulta Global de Reservas del Sistema

- **Ruta de Acceso:** GET `/pistaPadel/admin/reservations`

- **Seguridad y Permisos:** Operación restringida exclusivamente para usuarios con rol `ROLE_ADMIN`.
- **Lógica Funcional:** Recupera la totalidad de los registros de reserva guardados en la base de datos sin aplicar filtros de propiedad personal. Incluye las relaciones hidratadas de la entidad Pista y la entidad Usuario para dar soporte al panel de control global del administrador.
- **Códigos de Estado HTTP Devueltos:**
 - 200 OK: Éxito en la descarga masiva de datos para la gestión administrativa.
 - 403 Forbidden: Denegación de acceso si un usuario ordinario intenta listar las reservas de otros clientes.
- **Ejemplo de Trazabilidad y Captura:**



Endpoint 10: Cancelación de Reserva Existente

- **Ruta de Acceso:** `DELETE /pistaPadel/reservations/{reservationId}`
- **Seguridad y Permisos:** Permitido al usuario propietario de la reserva o a cualquier usuario con rol `ROLE_ADMIN`.
- **Lógica Funcional:** Evalúa la existencia del registro. El servicio comprueba si el usuario que solicita la cancelación coincide con el titular de la reserva o si posee rango de administrador. En caso afirmativo, se procede a la eliminación del registro, liberando de inmediato las franjas horarias ocupadas en la base de datos.
- **Códigos de Estado HTTP Devueltos:**
 - 24 No Content: Eliminación de la reserva completada satisfactoriamente.

- 401 Unauthorized: Intento de manipulación de una reserva ajena sin los privilegios requeridos.

3.6.4. Endpoints del Bloque de Autenticación y Gestión de Usuarios

Endpoint 11: Registro de Nuevo Usuario

- **Ruta de Acceso:** POST /pistaPadel/auth/register
- **Seguridad y Permisos:** Endpoint público. No requiere token de autenticación previo en las cabeceras.
- **Estructura del Cuerpo de la Petición (Request Body):**

```
{
  "nombre": "Juan",
  "apellidos": "Pérez Gómez",
  "email": "juan.perez@example.com",
  "telefono": "600123456",
  "password": "Password123"
}
```

- **Lógica Funcional y Validaciones:** El controlador recibe los datos de inscripción y valida la sintaxis mediante anotaciones Bean Validation (@Email, @NotBlank, etc.). El servicio comprueba a través del repositorio si el correo electrónico ya se encuentra registrado de forma insensible a mayúsculas (existsByEmailIgnoreCase). Si existe una coincidencia, se interrumpe el flujo lanzando una excepción de negocio por conflicto. Las contraseñas válidas se procesan y se asigna por defecto el rol operativo ROLE_USER en la persistencia inicial.
- **Códigos de Estado HTTP Devueltos:**
 - 201 Created: Registro completado correctamente. Retorna el perfil consolidado del nuevo usuario omitiendo la clave por seguridad.
 - 400 Bad Request: Parámetros de entrada inválidos, sintaxis de correo electrónica errónea o campos requeridos ausentes.
 - 409 Conflict: El email suministrado ya está asignado a otra cuenta activa en el sistema.
- **Ejemplo de Trazabilidad y Captura:**

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8080/pistaPadel/auth/register`
- Method:** `POST`
- Body (raw):**

```

1 {
2   "nombre": "Martina",
3   "apellidos": "Ortiz",
4   "email": "martina@gmail.com",
5   "password": "123456",
6   "telefono": "987"
7 }

```
- Response (JSON):**

```

1 {
2   "id": 6,
3   "nombre": "Martina",
4   "apellidos": "Ortiz",
5   "email": "martina@gmail.com",
6   "telefono": "987",
7   "rol": "USER",
8   "fechaRegistro": "2026-05-17T22:24:16.2692977",
9   "activo": true
10 }

```
- Status:** 201 Created, 167 ms, 570 B

Endpoint 12: Autenticación e Inicio de Sesión (Login)

- **Ruta de Acceso:** `POST /pistaPadel/auth/login`
- **Seguridad y Permisos:** Endpoint público. Acceso libre sin credenciales en la cabecera.
- **Estructura del Cuerpo de la Petición (Request Body):**

```

{
  "email": "juan.perez@example.com",
  "password": "Password123"
}

```

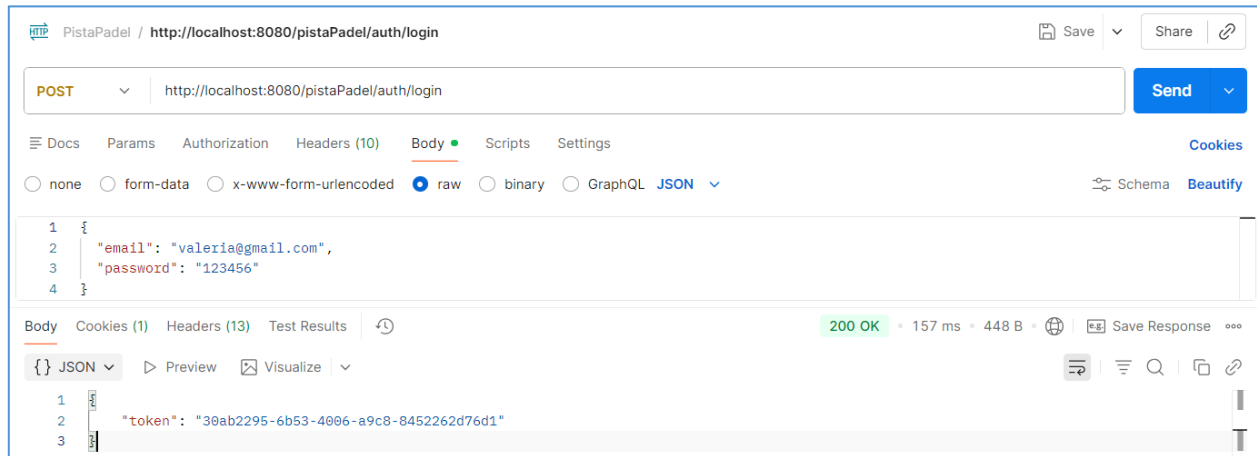
- **Estructura de la Respuesta (Response Body):**

```

{
  "userId": 5,
  "userRol": "USER",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI1IiwiaXNjaXkiOiJjaXhwIjoxNzY..."
}

```

- **Lógica Funcional:** Autentica la identidad del cliente evaluando el par credencial provisto. El servicio localiza la entidad por el email de entrada y compara la contraseña. Al comprobarse la integridad del acceso, el subsistema criptográfico genera un token firmado digitalmente que empaqueta las propiedades de identidad y el rol jerárquico del usuario, devolviendo el DTO unificado que se almacenará en el cliente.
- **Códigos de Estado HTTP Devueltos:**
 - 200 OK: Credenciales válidas. Devuelve el token criptográfico y los metadatos esenciales de sesión.
 - 401 Unauthorized: El correo no existe o la contraseña introducida no coincide con los registros.
- **Ejemplo de Trazabilidad y Captura:**

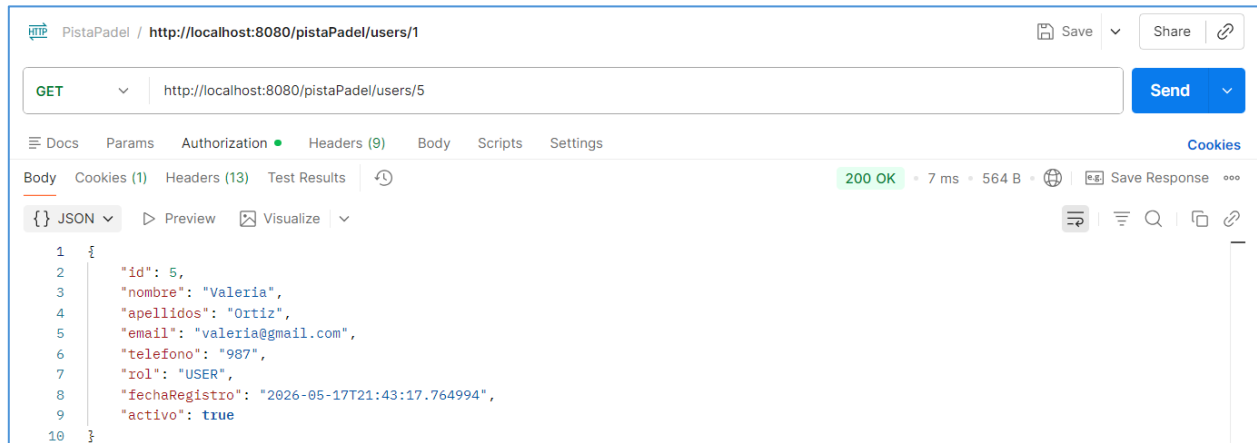


Endpoint 13: Consulta y Extracción del Perfil de Usuario

- **Ruta de Acceso:** GET /pistaPadel/users/{userId}
- **Seguridad y Permisos:** Requiere autenticación por token JWT en cabecera. Restringido al propio usuario dueño del recurso o a cuentas con privilegios de nivel ROLE_ADMIN.
- **Estructura de la Respuesta (Response Body):**

```
{ 1: { 2:   "id": 5, 3:   "nombre": "Juan", 4:   "apellidos": "Pérez Gómez", 5:   "email": "juan.perez@example.com", 6:   "telefono": "600123456", 7:   "userRol": "USER" 8: }
```

- **Lógica Funcional:** Recupera el identificador numérico de la ruta y evalúa el token inyectado en el contexto de seguridad para asegurar la correspondencia del firmante. El componente de negocio interroga al repositorio mediante findById(), transformando la entidad JPA interna en un objeto de respuesta filtrado (DTO) que remueve el hash criptográfico de la contraseña para impedir fugas de información sensible sobre la red.
- **Códigos de Estado HTTP Devueltos:**
 - 200 OK: Petición atendida satisfactoriamente con la información del perfil en el cuerpo de la respuesta.
 - 401 Unauthorized: Intento de lectura sin la cabecera de autenticación Bearer válida.
 - 403 Forbidden: Token legítimo pero el usuario intenta consultar de manera cruzada un perfil ajeno sin rango de administración.
 - 404 Not Found: El identificador de usuario provisto no existe en la persistencia del sistema.
- **Ejemplo de Trazabilidad y Captura:**



Endpoint 14: Modificación y Actualización de Perfil

- **Ruta de Acceso:** PATCH /pistaPadel/users/{userId}
- **Seguridad y Permisos:** Requiere token válido. Exclusivo para el titular de la cuenta o usuarios con el rol ROLE_ADMIN.
- **Estructura del Cuerpo de la Petición (Request Body):**

```
{
  "nombre": "Juan Ignacio",
  "apellidos": "Pérez Gómez",
  "email": "juan.perez@example.com",
  "telefono": "655999888",
  "password": "NuevaPassword123"
}
```

- **Lógica Funcional:** Extrae el registro original de la base de datos H2 para servir de base operativa. El servicio mapea las propiedades editables provistas en el cuerpo e implementa validaciones cruzadas para evitar colisiones de email con terceros registros. Si el campo de contraseña contiene una nueva cadena, el sistema aplica la lógica de cifrado antes de consolidar el cambio mediante la interfaz de persistencia.
- **Códigos de Estado HTTP Devueltos:**
 - 200 OK: Modificación estructural confirmada y persistida de forma segura en la base de datos relacional.
 - 400 Bad Request: Estructura de datos inconsistente, teléfonos mal formados o cadenas obligatorias vacías.
 - 403 Forbidden: Intento de manipulación de registros de terceros por parte de un rol no autorizado.
 - 404 Not Found: El identificador de ruta no se asocia con ninguna entidad real del sistema.

Endpoint 15: Cierre de Sesión Remoto (Logout)

- **Ruta de Acceso:** POST /pistaPadel/auth/logout
- **Seguridad y Permisos:** Requiere autenticación de usuario (ROLE_USER o ROLE_ADMIN).

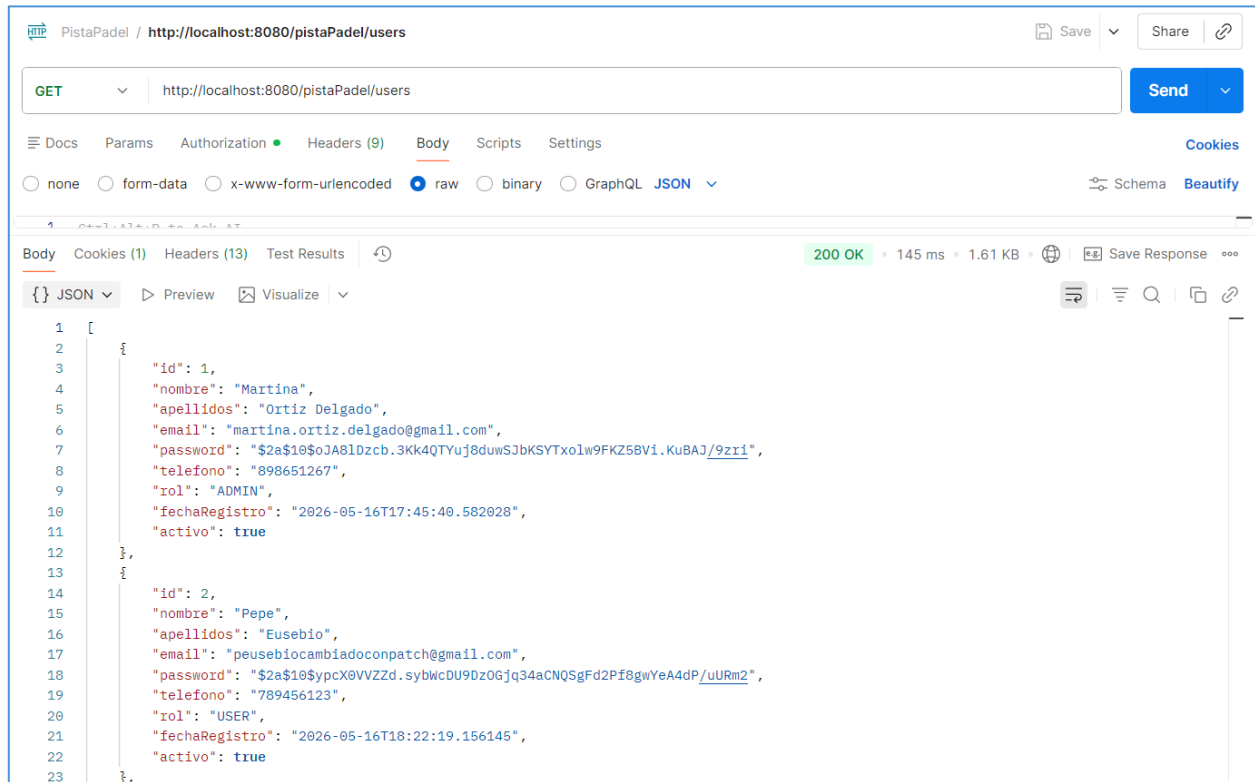
- **Lógica Funcional:** Recibe e intercepta el token Bearer activo en la petición HTTP. El servicio procesa la solicitud de salida e invalida la firma criptográfica en el ámbito del backend notificando al cliente la correcta desconexión, facilitando que el frontend purgue los datos locales guardados de forma segura.
- **Códigos de Estado HTTP Devueltos:**
 - 200 OK: Sesión cerrada con éxito. El estado en el cliente se actualiza y redirige a la pantalla de login.
 - 401 Unauthorized: Petición de desconexión sin suministrar un token de portador estructurado.

Endpoint 16: Recuperación Masiva del Inventario de Usuarios

- **Ruta de Acceso:** GET /pistaPadel/users
- **Seguridad y Permisos:** Autenticación JWT requerida. Acceso restringido de forma estricta a usuarios con rol operativo ROLE_ADMIN.
- **Estructura de la Respuesta (Response Body):**

```
[
  {
    "id": 1,
    "nombre": "Admin",
    "apellidos": "ChisPadel",
    "email": "admin@chispadel.com",
    "telefono": "911111111",
    "userRol": "ADMIN"
  },
  {
    "id": 5,
    "nombre": "Juan",
    "apellidos": "Pérez Gómez",
    "email": "juan.perez@example.com",
    "telefono": "655999888",
    "userRol": "USER"
  }
]
```

- **Lógica Funcional:** Endpoint diseñado para alimentar el Panel de Control Administrativo (admin.js). Ejecuta una consulta masiva no filtrada a la tabla de usuarios de la base de datos a través de Spring Data JPA, mapeando la colección de salida hacia DTOs limpios sin contraseñas para ofrecer una renderización fluida en forma de tabla HTML.
- **Códigos de Estado HTTP Devueltos:**
 - 200 OK: Descarga de datos masiva completada con éxito para la gestión y control de personal del club.
 - 403 Forbidden: Intento de acceso denegado por falta de privilegios de nivel administrativo.
- **Ejemplo de Trazabilidad y Captura:**



3.7. Tareas programadas y notificaciones

Se describirá la implementación de las dos tareas automáticas: el recordatorio diario de reservas y el informe mensual de disponibilidad.

La automatización de procesos en el servidor de aplicaciones se gestiona mediante el módulo de tareas programadas nativo de Spring, habilitado a través de la anotación `@EnableScheduling` en la clase de configuración principal. Estas rutinas operan de manera asíncrona en hilos de ejecución en segundo plano, evitando penalizar el rendimiento de los endpoints de la API expuestos a los usuarios.

En primer lugar, se crea `remindPista()`, cuyo objetivo es enviar un correo electrónico para recordar a los usuarios que tienen una pista reservada para ese mismo día.

```
@Scheduled(cron = "0 0 2 * * *") // En el segundo y minuto 0, a las 2am de cada día, mes.
public void remindPista() {
    logger.info("Me ejecuto cada día a las 2 AM");

    /* Mandar correo a usuarios que tienen pista reservada para ese día */

    servicioReserva.enviarRecordatorioDia(); // Esta clase debemos implementarla en Servicio
}
```

En segundo lugar, `showDisponibilidad()` genera y envía un informe mensual de disponibilidad de pistas a los usuarios, incorporando un manejo de excepciones ante posibles errores.

```

@Scheduled(cron = "0 0 0 1 * *") // Se ejecutará justo al empezar el día 1
public void showDisponibilidad() {
    logger.info("Tarea Programada → Ejecuto informe de disponibilidad mensual");

    try {
        servicioPista.enviarDisponibilidadMensual();
        logger.info("Disponibilidad mensual enviada correctamente a todos los usuarios.");
    } catch (Exception e) {
        logger.error("Error al enviar disponibilidad mensual: {}", e.getMessage());
    }
}

```

En ambas nos apoyamos en la anotación “@Scheduled”, la cual permite programar la ejecución automática en función de determinados intervalos de tiempo.

3.8. Tratamiento de errores y trazabilidad

Se documentará el subsistema de manejo centralizado de errores mediante @ControllerAdvice, la distinción rigurosa entre los códigos HTTP de error, y el sistema de trazas con SLF4J.

El subsistema de control y confinamiento de excepciones de *ChisPadel* está diseñado bajo un patrón centralizado, evitando la dispersión de bloques try-catch redundantes en la capa de controladores y garantizando respuestas con estructuras semánticas estandarizadas hacia los clientes de la API REST.

Gestión Centralizada con @ControllerAdvice

La clase ManejadorErroresGlobales.java intercepta de manera transversal cualquier excepción no capturada en las capas inferiores de la aplicación. Al hacer uso de @ControllerAdvice, el componente evalúa la tipología del error y mapea la excepción hacia un código de respuesta HTTP coherente.

```

@ControllerAdvice

public class ManejadorErroresGlobales {

    @ResponseBody

    @ExceptionHandler(ResponseStatusException.class)

    public ResponseEntity errorLanzado(ResponseStatusException ex) {

        return new ResponseEntity<>(ex.getStatusCode());

    }
}

```

Distinción Rigurosa de Códigos de Estado HTTP

El servidor discrimina con precisión los fallos mediante la siguiente política de respuestas:

- **400 Bad Request:** Lanzado ante malformaciones sintácticas en los cuerpos JSON remitidos por el frontend o por violaciones en las restricciones de validación elementales.
- **401 Unauthorized:** Determinado por la capa de Spring Security ante firmas JWT corruptas, caducadas o nulas.
- **403 Forbidden:** Aplicado cuando las credenciales del usuario autenticado carecen de los privilegios o roles necesarios requeridos por el método invocado.
- **404 Not Found:** Producido cuando las consultas por clave primaria sobre recursos específicos (ej. /courts/999) resultan en colecciones vacías.
- **409 Conflict:** Utilizado para proteger la lógica de negocio frente a colisiones en la persistencia de datos (por ejemplo, registrar una pista con una denominación idéntica a una preexistente).

Sistema de Trazas e Instrumentación con SLF4J

La visibilidad del comportamiento interno de la aplicación en producción se consigue mediante el uso de la fachada de logging SLF4J conectada a la implementación de Logback. Se configuran tres niveles de traza:

- **INFO:** Registra los arranques del sistema, inicialización de tareas cron y accesos validados a los controladores.
- **DEBUG:** Imprime las queries internas o los payloads de las peticiones en fase de desarrollo para agilizar la depuración de la API.
- **ERROR:** Captura la traza de la pila (*stack trace*) ante excepciones imprevistas o fallos de conectividad con la base de datos relacional H2.

3.9. Estrategia de pruebas del backend

Para garantizar la robustez, estabilidad e integridad del sistema de gestión de reservas ChisPadel frente a cambios en el código o futuras ampliaciones, el equipo ha diseñado e implementado una estrategia de pruebas automatizadas en el backend. Esta batería de pruebas se divide en dos tipologías principales y complementarias —pruebas de integración de persistencia y pruebas de extremo a extremo (E2E)—, ejecutadas bajo una política estricta de aislamiento mediante la configuración de entornos controlados.

3.9.1. Pruebas de integración de la capa de repositorio con @DataJpaTest

El primer bloque de la batería se centra de forma exclusiva en la capa de persistencia y el acceso a datos. Estas pruebas se implementan en la clase `IntegrationTest` haciendo uso de la anotación especializada `@DataJpaTest` de Spring Boot Test.

La ventaja fundamental de `@DataJpaTest` radica en que deshabilita la configuración completa de la aplicación y escanea únicamente las entidades anotadas con `@Entity` y las interfaces de repositorio de Spring Data JPA. Esto acelera significativamente la velocidad de ejecución y delimita el alcance de las pruebas a la base de datos relacional H2 en memoria.

Entre los aspectos críticos validados mediante esta aproximación destacan:

- **Mantenimiento y consistencia de relaciones lógicas:** Se verifica que al almacenar una entidad compleja (como Reserva), Spring Data JPA e Hibernate hidrotén y enlacen correctamente las claves ajenas físicas (usuario_id y pista_id) con sus respectivas entidades persistidas (Usuario y Pista), validando que las propiedades de navegación no resulten nulas.
- **Cumplimiento de restricciones de integridad:** Se comprueba el comportamiento del motor ante la inserción de registros duplicados bajo campos declarados únicos, verificando que el repositorio lance excepciones controladas de persistencia como DataIntegrityViolationException cuando se intenta forzar, por ejemplo, el registro de dos usuarios con el mismo correo electrónico.

3.9.2. Pruebas de extremo a extremo (E2E) con @SpringBootTest

El segundo bloque metodológico evalúa el comportamiento holístico del backend a través de pruebas de extremo a extremo (E2E) implementadas en la clase ControladorRestE2ETest. A diferencia del enfoque de persistencia aislado, aquí se emplea la anotación @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT). Esta configuración levanta el contexto de Spring de forma completa, incluyendo todas sus capas constitutivas (filtros de seguridad, controladores REST, servicios lógicos de negocio y repositorios de datos) y arranca un servidor web real en un puerto local asignado aleatoriamente. La comunicación y simulación de peticiones desde el exterior hacia la API se articula mediante el cliente especializado TestRestTemplate.

A través de esta metodología se validan flujos transaccionales reales, tales como:

- **El ciclo completo de vida de una petición HTTP:** Desde la interceptación de la cabecera en los filtros, pasando por las validaciones de beans (@Valid), la lógica transaccional de los servicios (por ejemplo, cálculos de exclusión de intervalos horarios en las reservas) hasta la serialización automática de la entidad de salida a formato JSON.
- **La asertividad en las respuestas HTTP y contratos REST:** Se contrastan de forma estricta los códigos de estado devueltos (200 OK, 201 Created, 404 Not Found, etc.) ante peticiones exitosas o forzadas a fallar, asegurando que la API se comporte con total fidelidad respecto al contrato especificado en la documentación de endpoints.

3.9.3. Estrategia de aislamiento mediante perfiles y @DirtiesContext

Uno de los principales riesgos en las baterías de pruebas automatizadas es el acoplamiento o la contaminación del estado de la base de datos entre ejecuciones sucesivas, lo que puede originar falsos positivos o fallos en cascada. Para erradicar este problema, el equipo ha adoptado una estrategia basada en dos mecanismos clave de Spring Framework:

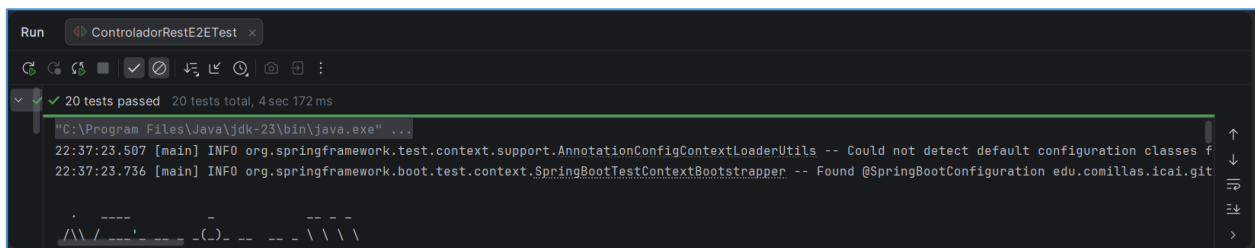
1. **Uso de perfiles activos (@ActiveProfiles("test")):** Mediante esta anotación, se segrega por completo el entorno de ejecución de las pruebas respecto al de desarrollo o producción. Esto permite mapear la aplicación a un archivo de propiedades exclusivo (application-test.properties), configurando una base de datos H2 volátil e independiente que se recrea limpia al inicio de los tests, sin alterar los datos estables de trabajo del equipo.
2. **Reinicialización del contexto con @DirtiesContext:** En escenarios donde una prueba realiza mutaciones estructurales o escrituras pesadas sobre la persistencia que comprometen los casos de prueba siguientes, se añade la anotación @DirtiesContext. Esta instrucción indica explícitamente a la suite de JUnit que el contexto de Spring ha quedado "sucio" tras la ejecución del método o clase de prueba. En consecuencia, Spring

cierra el contexto actual, vacía la base de datos en memoria y levanta una instancia del servidor completamente nueva desde cero para el siguiente test, garantizando un aislamiento y una repetibilidad absoluta en toda la suite.

A continuación, se muestra una prueba visual del funcionamiento de ambos tests, tanto el de integración como del E2E:



```
Run IntegrationTest x
25 tests passed 25 tests total, 1 sec 279 ms
"C:\Program Files\Java\jdk-23\bin\java.exe" ...
22:36:40.616 [main] INFO org.springframework.test.context.support.AnnotationConfigContextLoaderUtils -- Could not detect default configuration classes for class [org.springframework.test.context.support.AnnotationConfigContextLoaderUtils]
22:36:40.811 [main] INFO org.springframework.boot.test.context.SpringBootTestContextBootstrapper -- Found @SpringBootConfiguration edu.comillas.icaigit
```



```
Run ControladorRestE2ETest x
20 tests passed 20 tests total, 4 sec 172 ms
"C:\Program Files\Java\jdk-23\bin\java.exe" ...
22:37:23.507 [main] INFO org.springframework.test.context.support.AnnotationConfigContextLoaderUtils -- Could not detect default configuration classes for class [org.springframework.test.context.support.AnnotationConfigContextLoaderUtils]
22:37:23.736 [main] INFO org.springframework.boot.test.context.SpringBootTestContextBootstrapper -- Found @SpringBootConfiguration edu.comillas.icaigit
```

4. DESARROLLO DE LA INTERFAZ DE USUARIO (FRONTEND)

4.1. Arquitectura y Diseño Estético Global

La interfaz de usuario de *ChisPadel* se ha realizado bajo los principios de diseño web responsivo, usabilidad y separación limpia de responsabilidades (HTML5 estructural, CSS3 declarativo y JavaScript nativo guiado por eventos, utilizando la API *Fetch* para la comunicación asíncrona con el servidor).

El diseño visual se encuentra centralizado en un único archivo de estilos (*index.css*), el cual establece un sistema de diseño consistente mediante el uso de variables CSS (*Custom Properties*). Esto asegura que cualquier modificación en la paleta cromática o en la tipografía se propague de manera idéntica por todas las vistas de la aplicación.

```
/* Fragmento de la hoja de estilos declarativa centralizada index.css */
:root {
  --azul-oscuro: rgba(46, 114, 157);
  --azul-medio-oscuro: rgba(53, 127, 166);
  --azul: rgba(74, 165, 195);
  --azul-claro: rgba(205, 232, 243);
  --verde-claro: rgba(179, 238, 157);
  --verde-lima: rgba(216, 245, 62);
  --verde-oscuro: rgba(144, 205, 49);
  --amarillo: rgba(254, 222, 98);
  --naranja: rgba(252, 181, 60);
  --blanco: rgba(254, 254, 254);
}

body {
  font-family: 'Poppins', sans-serif;
  background-color: var(--blanco);
  color: var(--azul-oscuro);
}
```

La identidad corporativa utiliza tonos azulados para aportar profesionalidad y confianza en las secciones de gestión de datos, combinados con sutiles toques verde lima y naranja para evocar el entorno deportivo del pádel y destacar los elementos interactivos (*Call to Action*), tales como botones de confirmación y estados de disponibilidad activa.

4.2. Arquitectura de Integración y Ciclo de Vida del Token de Sesión

El frontend de *ChisPadel* se ha diseñado bajo el paradigma de una interfaz orientada a eventos y desacoplada del servidor analítico, consumiendo los endpoints expuestos por la API REST de Spring Boot a través de peticiones HTTP asíncronas mediante la API nativa *Fetch*. La persistencia de la sesión del usuario en el entorno del cliente no se delega en cookies tradicionales para mitigar vulnerabilidades de tipo *Cross-Site Request Forgery* (CSRF); en su lugar, se implementa almacenamiento local aislado a través de la interfaz *localStorage* del navegador web.

El ciclo de vida de la autenticación se rige por un flujo de intercambio de credenciales JSON que, tras ser validadas satisfactoriamente por la lógica de negocio del backend, retorna un objeto con el identificador numérico único (*userId*), el rol operativo (*userRol*) y el token criptográfico. Este token actúa como firma de identidad y se inyecta dinámicamente en las cabeceras HTTP mediante el esquema de autenticación estándar *Bearer* para validar el acceso a los recursos protegidos:

```
const token = localStorage.getItem("token");
const headers = {
  "Content-Type": "application/json",
  ...(token && { "Authorization": `Bearer ${token}` })
};
```

Este mecanismo garantiza que el servidor pueda interceptar, decodificar y validar la firma en cada petición, abstrayendo al cliente de la lógica interna de sesión y manteniendo un estado *stateless* en el backend.

4.2.1. Lógica de Sincronización Asíncrona del Lado del Cliente (usuario.js)

El script usuario.js centraliza la captura de eventos de los formularios de acceso, el registro de nuevos usuarios en el sistema y la mutación del perfil personal. Su lógica operativa se basa en la interceptación sistemática del método `event.preventDefault()` sobre los escuchas de eventos (`addEventListener`). Con esto se bloquea el refresco por defecto de los elementos `<form>`, permitiendo que JavaScript asuma el control total del hilo de ejecución y realice el envío asíncrono en segundo plano:

```
formLogin?.addEventListener("submit", async function (event) {
  event.preventDefault();

  const email = document.getElementById("email").value;
  const password = document.getElementById("password").value;

  const response = await fetch(`${baseUrl}/pistaPadel/auth/login`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email, password })
  });
  // ... procesamiento de la respuesta y almacenamiento de credenciales
});
```

Como medida preventiva de optimización la lógica del cliente realiza una validación de integridad local simétrica en el formulario de registro y edición para verificar que la confirmación de la contraseña coincide con el campo original antes de estructurar el objeto JSON de transporte. Si esta regla falla, la ejecución se interrumpe de inmediato y se invoca a la función de feedback visual, protegiendo al servidor de peticiones innecesarias o defectuosas.

The image displays two versions of a user registration form side-by-side. Both forms have a light blue background and white input fields. The left form shows the state after an invalid submission. The inputs for 'Nombre' (Atllano), 'Apellidos' (Fernández), 'Email' (afernandez@gmail.com), 'Teléfono' (123456789), 'Contraseña' (masked with dots), and 'Confirmar contraseña' (masked with dots) are visible. A red error message 'Datos inválidos' is displayed below the form. The right form shows the state after a successful submission. The inputs are the same, but the 'Contraseña' and 'Confirmar contraseña' fields are now filled with asterisks. A green success message 'Usuario creado correctamente' is displayed below the form. Both forms have a green 'Crear cuenta' button and a link '¿Ya tienes cuenta? Inicia Sesión' at the bottom.

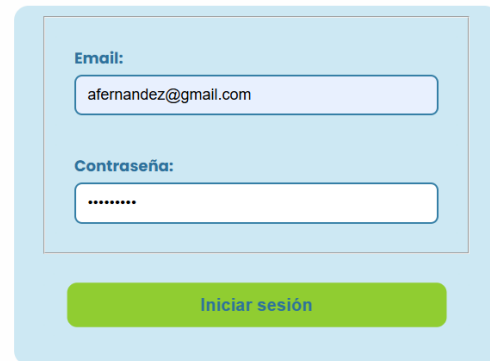


Email:
afernandez@gmail.com

Contraseña:

Iniciar sesión

Email o contraseña incorrectos



Email:
afernandez@gmail.com

Contraseña:

Iniciar sesión

¡Bienvenido!



Editar perfil

Nombre:
Atilano

Apellidos:
Fernández

Email:
afernandez@outlook.com

Teléfono:
123456789

Nueva contraseña (dejar vacío para no cambiar):

Confirmar nueva contraseña:

Guardar cambios

Perfil actualizado correctamente

Cerrar sesión

Al cargarse diferidamente el árbol DOM (DOMContentLoaded), el script determina si la vista actual requiere la extracción de información persistida. En la interfaz de perfil, opera de manera bidireccional: inyecta nodos de texto plano (textContent) en las etiquetas estéticas de resumen y, simultáneamente, mapea de forma inversa esos valores hacia los atributos .value de los cuadros de inserción de datos. Esta sincronización asegura que cualquier posterior mutación de datos mediante la petición PATCH /pistaPadel/users/{userId} mantenga como base el estado real exacto de la base de datos relacional H2, impidiendo la sobreescritura accidental de atributos nulos o campos vacíos.




Bienvenido, Atilano Fernández

Email: afernandez@gmail.com

Teléfono: 123456789

Rol: USER

Miembro desde: 2026-05-17

8	TRUE	Fernández	afernandez@gmail.com	2026-05-17 22:52:24.449211	Atilano	\$2a\$10\$089vuZtr.NIC.E0wM2LPN.Y7LPmoOIFZSj6EKA9gigKEL2.n1aPhu	USER	123456789
8	TRUE	Fernández	afernandez@outlook.com	2026-05-17 22:52:24.449211	Atilano	\$2a\$10\$089vuZtr.NIC.E0wM2LPN.Y7LPmoOIFZSj6EKA9gigKEL2.n1aPhu	USER	123456789

4.2.2. Panel Administrativo Global y Generación Estructural de Listados (admin.js)

Para dar cumplimiento a las especificaciones relativas a los perfiles de administración estipulados en los requisitos funcionales del sistema, el archivo admin.js automatiza de manera exclusiva la extracción remota del inventario de cuentas de usuario y acopla la información en una tabla integrada con las directrices visuales de la hoja de estilos global “index.css”, quedando resumidas en sus filas, todas las posibles acciones que puede realizar un usuario administrador.

En esta tabla, se reunirán las acciones ligadas a varios repositorios (una función por cada botón) y se accederá a ella, una vez el usuario ADMIN ha realizado el “login” o bien, clicando en “Mi Perfil”.

Además, el tratado de esta será dinámico. Cuando se pulse el botón “Ejecutar”, se ocultará la tabla para que se efectúe la acción; dando al usuario la posibilidad de regresar hacia atrás en caso de no querer realizar dicha acción sobre la base de datos.

Panel de Administración		
Función	Descripción	Acceso
Cargar usuarios	Obtener listado completo de usuarios	Ejecutar
Cargar usuario por ID	Buscar un usuario específico	Ejecutar
Editar usuario por ID	Modificar datos de un usuario	Ejecutar
Crear pista	Registrar una nueva pista	Ejecutar
Editar pista por ID	Modificar pista	Ejecutar
Eliminar pista por ID	Eliminar pista del sistema	Ejecutar
Ver reservas	Consultar reservas realizadas	Ejecutar
Ver reserva por ID	Consultar reserva concreta realizada	Ejecutar
Borrar reserva por ID	Borrar reserva concreta realizada	Ejecutar
Editar reserva por ID	Editar reserva concreta realizada	Ejecutar

En cuanto al control preventivo de privilegios cruzados, este se gestiona dentro del propio JS analizando el código de estado HTTP de la respuesta de la API. Si un usuario no autenticado o sin el rol adecuado intentara saltarse las restricciones visuales forzando el acceso directo a la vista admin.html, la subrutina interceptará el rechazo emitido por los filtros de seguridad del servidor y ejecutará una rutina de invalidación de sesión:

```
if (response.status === 403) {
    mostrarMensaje("Privilegios insuficientes para la lectura del recurso", "error");
    localStorage.clear();
    window.location.href = "login.html";
    return;
}
```

Una vez validada la respuesta satisfactoria (200 OK), el algoritmo procede al renderizado de la página. Durante este proceso, se evalúa el campo jerárquico del usuario para aplicar clases CSS dinámicas contextuales (como disp-ocupada para cuentas con rol ADMIN y disp-disponible para usuarios estándar USER). Esto permite al administrador discriminar visualmente el nivel operativo del personal directamente desde el cuadro de mandos, garantizando una interfaz fluida, responsiva y completamente guiada por el estado de los datos del backend.

A continuación, se muestran algunas pruebas visuales de la parte de administración para usuarios (aunque su funcionalidad se extienda también para pistas y reservas):

Buscar usuario por ID

Datos del usuario

ID: 1

Nombre: Martina Ortiz Delgado

Email: martina.ortiz.delgado@gmail.com

Teléfono: 898651267

Rol: ADMIN

Miembro desde: 2026-05-16

Activo: Si

Reservas del usuario

Sin reservas registradas.

Rol: ADMIN

Miembro desde: 2026-05-16

Cargar usuarios

ID	Nombre	Email	Rol
1	Martina Ortiz Delgado	martina.ortiz.delgado@gmail.com	ADM
2	Pepe Eusebio	peusebio@gmail.com	USE
3	Valeria Ortiz	valeria@gmail.com	ADM

ID	ACTIVO	APELLIDOS	EMAIL	FECHA_REGISTRO	NOMBRE	PASSWORD	ROL	TELEFONO
1	TRUE	Ramos	maria@gmail.com	2026-05-13 13:27:37.772404	Maria	\$2a\$10\$VtuO5NOYipbyQT9VP7uHYOMXInCnhEIVkJPWFfi4AUbuJ9OCs4XnC	USER	123456789
2	TRUE	Ramos	admin@chispadel.com	2026-05-13 13:51:05.342705	Ana	\$2a\$10\$AHHjOenKp7dsHg7yXJGpTumkt.L2M3LppTRF6kYIdUJ3njXHrVSe	ADMIN	666666666

Editar usuario por ID

Editar usuario por ID

Modificar datos

Re llena solo los campos que quieras cambiar.

Nombre

Apellidos

Email

Teléfono

Nueva contraseña (opcional)

Confirmar contraseña

ID	ACTIVO	APELLIDOS	EMAIL	FECHA_REGISTRO	NOMBRE	PASSWORD	ROL	TELEFONO
1	TRUE	Ramos	maria@gmail.com	2026-05-13 13:27:37.772404	Maria	\$2a\$10\$VtuO5NOYipbyQT9VP7uHYOMXInCnhEIVkJPWFfi4AUbuJ9OCs4XnC	USER	123456789
2	TRUE	Ramos	admin@chispadel.com	2026-05-13 13:51:05.342705	Ana	\$2a\$10\$AHHjOenKp7dsHg7yXJGpTumkt.L2M3LppTRF6kYIdUJ3njXHrVSe	ADMIN	666666666

4.3. Módulo de Gestión de Pistas

La administración de la infraestructura del club recae sobre el módulo de gestión de pistas. Este componente ofrece una doble interfaz: un panel de control avanzado restringido exclusivamente a usuarios con rol ADMIN, y una interfaz de consulta pública accesible por cualquier usuario autenticado de la plataforma.

4.3.1. Panel de Administración y Mutación de Pistas (pista.html y pista.js)

Cuando un usuario inicia sesión con privilegios de administrador, se habilita una barra de tareas dedicada en el archivo admin.html. Al accionar el botón de ejecución sobre la fila de gestión de pistas, el flujo de navegación redirige de forma segura al archivo pista.html.

Esta vista contiene un formulario dinámico que actúa de forma polimórfica como interfaz de creación, modificación y borrado lógico/físico de pistas. El archivo pista.js restringe el acceso en el ciclo de vida inicial (window.onload) validando la existencia del JSON Web Token (JWT) y el rol correspondiente en el almacenamiento local (*LocalStorage*).

```

window.onload = () => {
  if (!token) {
    alert("Please log in first");
    window.location.href = "login.html";
    return;
  }
  if (userRol !== "ADMIN") {
    alert("Access denied: Admin only");
    window.location.href = "index.html";
    return;
  }
  loadCourts();
  setupEvents();
};

```

El formulario se compone de un elemento `<select>` que recupera de forma asíncrona mediante un método GET la totalidad de las pistas existentes. Si el administrador mantiene la opción por defecto ("Selecciona una pista"), la lógica interpreta que se desea dar de alta una nueva instalación. En caso de que se intente registrar una pista cuyo identificador o nombre colisione con una ya existente en la base de datos, el backend deniega la operación devolviendo un código de estado 400 Bad Request o 409 Conflict, lo que desencadena un mecanismo de captura de excepciones en el frontend que alerta al usuario mediante un mensaje de veto explícito en la pantalla.

Si se selecciona una pista específica del desplegable, la función `onCourtSelect` realiza una precarga de los datos (*data binding* manual) rellenando los campos de texto: Nombre, Ubicación, Precio por hora y el estado de activación (booleano). Al pulsar el botón Guardar, el script discrimina el comportamiento: si existe un identificador seleccionado, efectúa una petición de mutación parcial mediante el método PATCH o total mediante PUT hacia la URL `/pistaPadel/courts/{courtId}`; de lo contrario, despacha un método POST.

To create a new pista:

1. choose crear nueva pista in the drop down menu Selecciona una pista.
2. Fill in nombre, ubicación, precio por hora, activa (true/false) and fecha de alta.
3. push guardar button.

To edit an existing pista:

1. choose the pista you want to edit in the drop down menu Selecciona una pista.
2. Edit the fields you want to change.
3. push guardar button.

To delete an existing pista:

1. choose the pista you want to delete in the drop down menu Selecciona una pista.
2. push eliminar pista button.

Pista - only ADMIN can change here

ID:

Selecciona una pista

Selecciona una pista

Crear nueva pista

Pista Real Central - Madrid - Centro Deportivo Norte

Pista Central - Madrid - Centro Deportivo Norte

Precio por hora:

Activa:

A continuación, se adjunta una prueba visual del funcionamiento de las validaciones ante la creación de dos pistas con nombre duplicado:

Pista – only ADMIN can change here

ID:

Selecciona una pista

Nombre:

Cheese

Ubicación:

Madrid

Precio por hora:

15

Activa:

true

Fecha de alta:

17/05/2026

Guardar

Eliminar pista

3. push guardar button.

Already a court with this name. Choose another name for the court :)

To delete an existing pista:
1. choose the pista you want to delete in the list
2. push eliminar pista button.

Aceptar

Pista – only ADMIN can change here

ID:

Selecciona una pista

Nombre:

Cheese

Ubicación:

PistaDuplicada

Precio por hora:

33

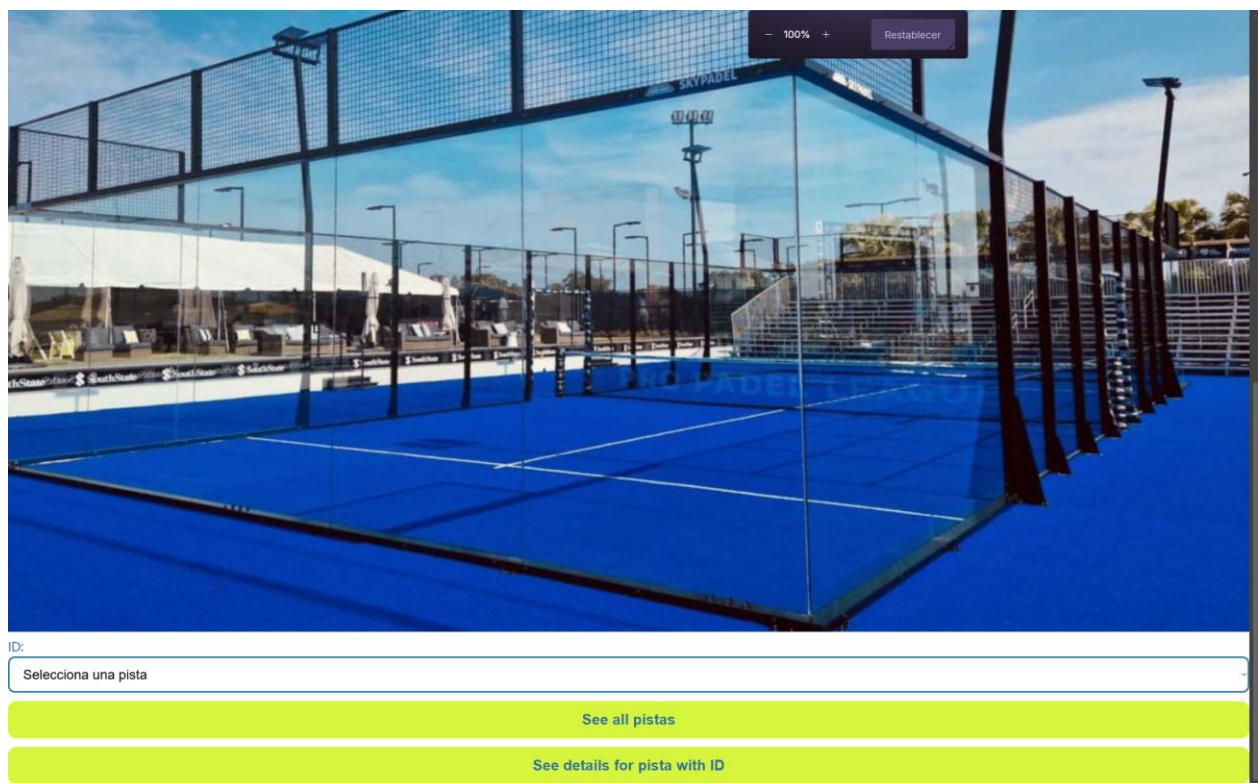
4.3.2. Vista de Catálogo e Información de Pistas (todosPistas.html y todosPistas.js)

Para los usuarios comunes que desean conocer los espacios del club sin capacidad de modificarlos, se ha diseñado la vista `todosPistas.html`. Al accionar el control "Ver pistas" de la página de inicio, el motor JavaScript invoca la función `seeAllCourts()`, la cual consume el endpoint público de lectura de pistas.

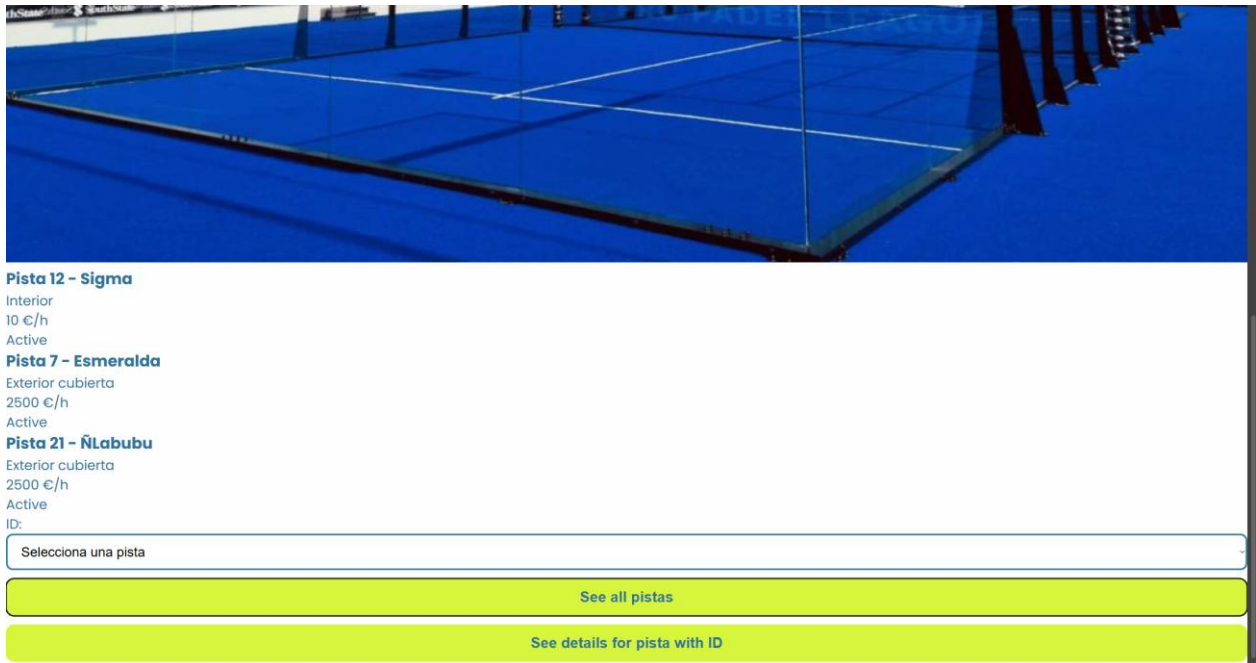
El script recorre la colección de objetos JSON devuelta por el servidor y construye dinámicamente un árbol de nodos en el DOM empleando plantillas literales de JavaScript (*Template Literals*). Cada pista se renderiza bajo un componente de tarjeta (.court-card), exponiendo de forma estilizada su denominación, localización geográfica y tarifa horaria. Asimismo, la vista incorpora un menú desplegable idéntico para aislar el análisis de una pista concreta; al ser seleccionado, se invoca a displayCourtDetails(court), que inyecta en un contenedor aislado (#courtDetails) el desglose minucioso de la entidad, incluyendo su fecha exacta de alta técnica en el sistema.



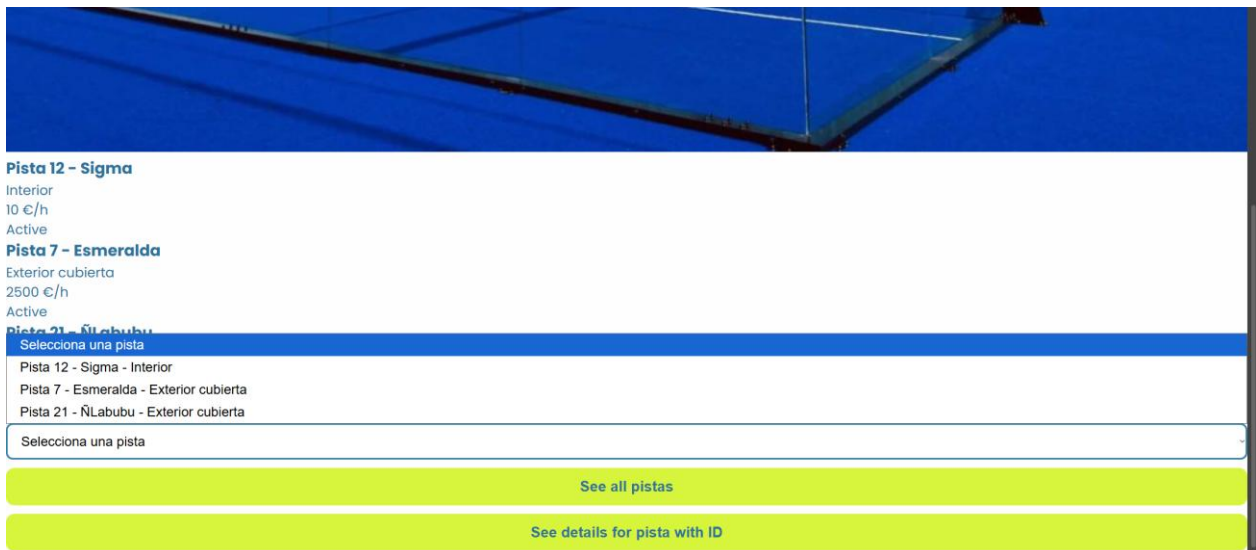
Para acceder a todosPistas.html se pulsa el botón “Ver pistas” en index.html.



Estado inicial de todosPistas.html.



Al pulsar el botón “See all pistas” se muestran todas las pistas con información básico.



Se muestra el menu desplegable de pistas de “Seleccionar una pista”.



Detalles de la pista

ID: 1
Nombre: Pista 12 - Sigma
Ubicación: Interior
Precio/hora: 10 €
Activa: Si
Fecha alta: 2026-05-17
 ID:

Pista 12 - Sigma - Interior

See all pistas

See details for pista with ID

Se muestra el resultado de pulsar el botón “See details for pista with ID” con la pista “Pista 12-Sigma-Interior” se obtiene información detallada de la pista seleccionada.

4.4. Módulo de Disponibilidad en Tiempo Real

4.4.1. Consulta Dinámica y Rejilla de Estados (disponibilidad.html y disponibilidad.js)

La vista disponibilidad.html presenta un cuadro de mando con un control de selección temporal de tipo `<input type="date">` y un filtro complementario opcional de pistas. El manejador de eventos asíncronos vinculado al botón de búsqueda ejecuta la función `mostrarResultados()`, abstrayendo la petición hacia el endpoint de composición de disponibilidad (`/pistaPadel/availability?date={fecha}`).

```
async function mostrarResultados() {

  const fecha = document.getElementById('fechaConsulta').value;
  const pistaId = document.getElementById('pistaConsulta').value;

  if (!fecha) {
    alert('Por favor selecciona una fecha');
    return;
  }

  let url = `${BASE_URL}/availability?date=${fecha}`;
  if (pistaId) url += `&courtId=${pistaId}`;

  try {
    const res = await fetch(url);
    if (!res.ok) throw new Error(res.status);
    const disponibilidades = await res.json();

    renderGrid(disponibilidades);
    document.getElementById('seccionTodasPistas').style.display = 'block';
    document.getElementById('seccionDetalle').style.display = 'none';
  }
}
```

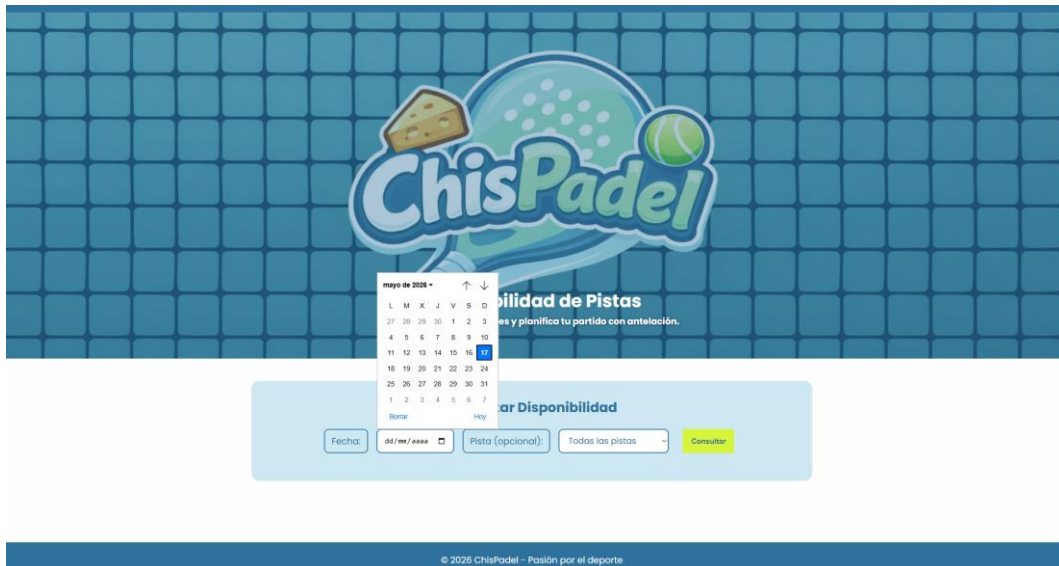
```

        document.getElementById('seccionTodasPistas').scrollIntoView({ behavior: 'smooth',
block: 'start' });
    } catch (err) {
        console.error('Error al consultar disponibilidad:', err);
        alert('Error al consultar la disponibilidad. ¿Está arrancado el servidor?');
    }
}

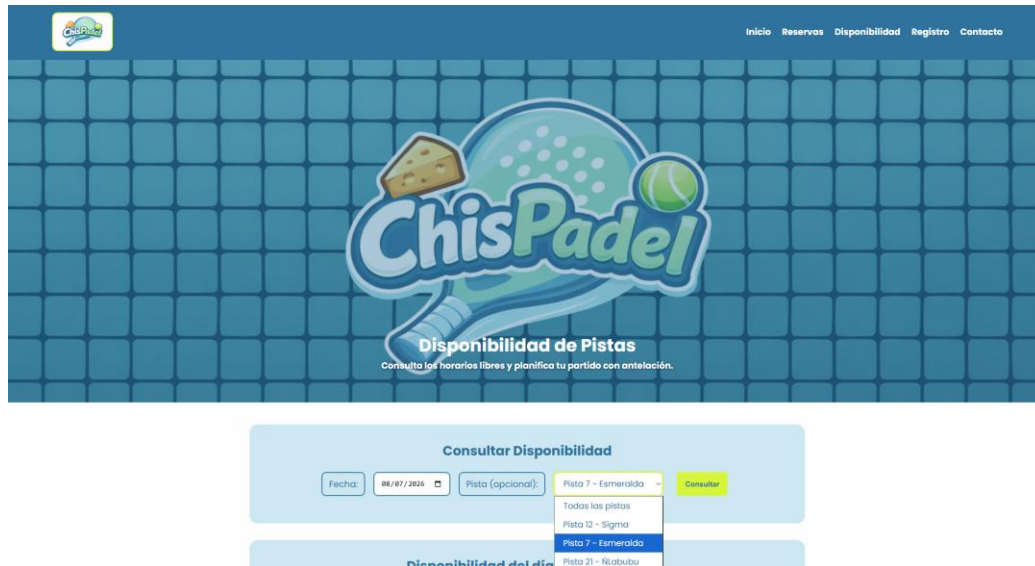
```

La respuesta del servidor consiste en un listado de estructuras de disponibilidad que contienen objetos anidados de la pista y colecciones de franjas horarias libres. La función `renderGrid` procesa operativamente este flujo de datos: limpia los contenedores previos y genera tarjetas estructurales.

Si una pista carece por completo de intervalos de tiempo desocupados para el día de la consulta, la lógica de la interfaz altera dinámicamente el estilo inyectando la clase CSS `.disponcupada` (que tiñe los bordes y etiquetas con tonos asociados a la indisponibilidad). Por el contrario, si existen horas vacantes, se aplica la clase `.disp-disponible` y se listan de forma resumida los bloques horarios iniciales.



Disponibilidad 1: Se muestra el menú desplegable Fecha en el que aparecen un calendario



Disponibilidad 2: Se muestra el menú desplegable Pista en el que aparecen todas las pistas que han sido creadas.



Disponibilidad 3: Se muestra el resultado de pulsar el botón “Consultar” con una fecha seleccionada y sin especificar pista, siendo que se muestran todas las pistas disponibles el 08/07/2026, en este caso solo está Disponible la “Pista7-Esmeralda” y se observa información de su disponibilidad, apertura, cierre, franjas libres y el botón “Ver detalle”.



Consultar Disponibilidad

Fecha: 18/05/2026 ☐ Pista (opcional): Todas las pistas

Disponibilidad del día seleccionado

Pista 12 - Sigma Ocupada

Apertura: 08:00
Cierre: 22:00

Franjas libres:
Sin franjas libres

Disponibilidad 4: Se muestra el resultado de pulsar el botón “Consultar” con una fecha seleccionada y sin especificar pista, siendo que se muestran todas las pistas disponibles el 18/05/2026, en este caso no hay franjas disponibles por lo que se muestra como ocupado.



Consultar Disponibilidad

Fecha: 08/07/2026 ☐ Pista (opcional): Pista 7 - Esmeralda

Disponibilidad del día seleccionado

Pista 7 - Esmeralda Disponible

Apertura: 09:00
Cierre: 23:00

Franjas libres:
11:00 – 13:00 18:30 – 20:30

Disponibilidad 5: Se muestra el resultado de pulsar el botón “Consultar” con una fecha seleccionada y con la pista “Pista 7-Esmeralda” seleccionada, siendo que se muestra la disponibilidad de dicha pista el 08/07/2026.

Disponibilidad 6: Se muestra el resultado de pulsar el botón “Consultar” con una fecha seleccionada y con la pista “Pista 12-Sigma” seleccionada, resultando en que al no haber disponibilidad dicho día para dicha pista se devuelva el mensaje “No hay datos de disponibilidad para esta fecha”.

4.4.2. Desglose Estructurado de Bloques Horarios (Vista de Detalle)

Dentro de cada tarjeta de disponibilidad se integra un botón con acción semántica de consulta profunda: Ver detalle. Al ser pulsado, se intercepta el identificador de la entidad y se despliega mediante transiciones suaves de CSS el contenedor #seccionDetalle, ocultando o minimizando la rejilla global según las necesidades del dispositivo del cliente.

Este panel realiza una inyección de datos de cabecera (#metaDetalle) donde expone las horas de apertura y cierre general de la instalación deportiva. Justo debajo, se formatea una tabla HTML (<table>) estructurada rígidamente en tres columnas: Hora de inicio, Hora de fin y Estado.

Si la pista cuenta con intervalos hábiles, el script itera sobre la propiedad franjasLibres, mapeando cada intervalo en una fila (<tr>) e insertando una etiqueta estilizada (Libre). En el supuesto de que el backend responda con un listado vacío para esa pista individual (escenario de saturación completa de reservas), el frontend intercepta la propiedad y genera una fila única con el atributo colspan="3", formateando un mensaje en cursiva y color naranja de advertencia que indica que no restan franjas utilizables.



Disponibilidad 7: Se muestra el resultado de pulsar el botón “Ver detalle”

4.5. Módulo de Transacciones de Reserva

4.5.1. Flujo de Usuario: Creación, Visualización y Reprogramación (reserva.html y reservas.js)

La interfaz reserva.html se divide funcionalmente en zonas lógicas para guiar al usuario a través del ciclo de vida de su reserva:

1. **Creación de Reservas:** El usuario interactúa con el formulario #crearReservaForm. Al inicializarse la vista, se ejecuta de manera transparente una llamada asíncrona hacia el backend para poblar el elemento <select> de pistas activas. El usuario escoge la instalación, la fecha y hora de inicio deseada y la duración del partido. Al procesarse el envío (*submit*), la función intercepta el evento, construye el cuerpo JSON con los tipos de datos requeridos por la API de Spring Boot y añade de forma obligatoria la cabecera de autenticación Authorization: Bearer <token>.

Crear Nueva Reserva

Pista:

Fecha y Hora de Inicio:

Fecha y Hora de Fin:

Crear Reserva

2. **Visualización de Historial Personal:** El componente recupera de forma automatizada las reservas exclusivas del usuario autenticado invocando al endpoint de autogestión del servidor. El JSON resultante es procesado para dar forma a la tabla #misReservasTabla. Cada fila incluye celdas con el código correlativo de la reserva, los metadatos de la pista, el momento temporal exacto y unos controles interactivos de acción.

Mis Reservas

Desde:

mm/dd/yyyy

Hasta:

mm/dd/yyyy

Filtrar

ID	Pista	Inicio	Fin	Duración (min)	Estado	Acciones
4	Pista Central	05/05/2026 22:18	06/05/2026 22:18	1440 min	ACTIVA	Editar Cancelar
5	Pista Central	18/05/2026 22:18	19/05/2026 22:18	1440 min	ACTIVA	Editar Cancelar
6	Pista Central	10/05/2026 22:18	11/05/2026 22:18	1440 min	ACTIVA	Editar Cancelar

Si el usuario es admin, en la misma página de reservas dispone de una segunda tabla de reservas sobre la cual también puede editar y cancelar reservas y que también dispone de un sistema de filtrado por fechas, pero incluye las reservas de todos los usuarios de la aplicación. Esta tabla de datos también es accesible mediante el panel de administración, lo cual se verá en detalle más adelante.

Todas las Reservas del Sistema

Desde: Hasta: Filtrar

ID	Usuario	Pista	Inicio	Fin	Duración (min)	Estado	Acciones
3	aaa	Pista Central	07/05/2026 13:45	08/05/2026 13:45	1440 min	ACTIVA	Editar Cancelar
4	ccc	Pista Central	05/05/2026 22:18	06/05/2026 22:18	1440 min	ACTIVA	Editar Cancelar
5	ccc	Pista Central	18/05/2026 22:18	19/05/2026 22:18	1440 min	ACTIVA	Editar Cancelar
6	ccc	Pista Central	10/05/2026 22:18	11/05/2026 22:18	1440 min	ACTIVA	Editar Cancelar

3. **Reprogramación y Cancelación:** Si una reserva se encuentra en un estado modificable (según las reglas de negocio del servidor), el usuario dispone de un botón para reprogramar. Esto activa el contenedor oculto #seccionReprogramar, rellenando el formulario con los datos preexistentes a fin de que el usuario altere únicamente la hora o la pista objetivo. Al guardar, se ejecuta una llamada de modificación HTTP. Asimismo, se provee un botón de cancelación que despacha un método DELETE, limpiando de forma inmediata la fila de la tabla tras la validación afirmativa en un cuadro de confirmación nativo.

Reprogramar Reserva

ID Reserva:

Pista:

Fecha y Hora de Inicio:

Fecha y Hora de Fin:

Guardar Cambios
Cancelar

4.5.2. Flujo de Administración: Panel de Supervisión Total de Reservas

Una de las ampliaciones más críticas de la lógica de negocio para el rol de administración consiste en la capacidad de auditar la base de datos de reservas en su totalidad. Para ello, el archivo admin.js se integra orgánicamente con las secciones ocultas de reserva.html.

Cuando el administrador accede a la función de carga global de reservas, el método `fetchTodasReservas()` realiza una petición REST hacia `/pistaPadel/reservas/all`. El frontend procesa esta respuesta masiva construyendo la tabla avanzada `#todasReservasTabla`. A diferencia de la vista de usuario, esta estructura incorpora una columna adicional que muestra el usuario asociado a cada reserva.

El administrador tiene también la posibilidad de acceder a métodos de búsqueda borrado y edición de reservas desde el panel de control empleando la id introducida para localizar la reserva sujeta dicha operación.

```
async function fetchTodasReservas() {
  try {
    const res = await fetch(`${baseUrlAdmin}/pistaPadel/reservas/all`, {
      headers: { "Authorization": "Bearer " + token }
    });
    if (!res.ok) throw new Error("Error en la descarga de datos");
    const reservas = await res.json();

    const tbody = document.getElementById("todasReservasTabla");
    tbody.innerHTML = reservas.map(r => `
      <tr>
        <td>${r.id}</td>
        <td>${r.usuario.email}</td>
        <td>${r.pista.nombre}</td>
        <td>${formatearFechaAdmin(r.fechaInicio)}</td>
        <td>${formatearFechaAdmin(r.fechaFin)}</td>
        <td>${r.duracionMinutos}</td>
        <td><span class="estado-confirmada">Activa</span></td>
        <td>
          <button class="btn-action-delete"
            onclick="abrirEliminarReserva(${r.id})">Eliminar</button>
        </td>
      </tr>
    `).join('');
    mostrarPanel("listados");
  } catch (err) {
    mostrarMensaje("No se pudo compilar el listado de reservas", "error");
  }
}
```

Además, tanto el panel de usuario como el de administrador integran una barra de filtrado temporal avanzada (`.filtro-fechas`). Al introducir un rango de fechas ("Desde" / "Hasta"), una rutina algorítmica local en JavaScript analiza el texto de la columna de tiempo de cada fila, realiza un parsing hacia objetos de tipo `Date` y computa una máscara de visibilidad alterando el estilo `display` de los nodos `<tr>`. El sistema ofrece un contador de registros visibles en tiempo real para agilizar la búsqueda de colisiones o la justificación de informes de ocupación.



5. CONCLUSIONES Y TRABAJO FUTURO

5.1. Conclusiones y Consolidación de Conocimientos

La finalización de este proyecto no solo representa la culminación de un trabajo en equipo enfocado en el desarrollo de una plataforma funcional, sino que constituye, fundamentalmente, un hito clave en la consolidación de los conocimientos teóricos y prácticos adquiridos a lo largo de la asignatura de Programación de Aplicaciones Telemáticas. El diseño e implementación de *ChisPadel* ha servido como el escenario idóneo para integrar un abanico muy amplio de tecnologías, metodologías y conceptos que, hasta el momento, se habían estructurado de forma fragmentada en las sesiones magistrales y de laboratorio. De este modo, la experiencia ha permitido transformar el aprendizaje académico en una competencia técnica real y aplicable al mundo profesional.

El Backend como Motor de Continuidad y Evolución

En el ámbito del *backend*, el desarrollo del sistema de reservas ha supuesto una valiosa oportunidad para retomar y fortalecer las bases de programación en Java que se introdujeron durante el curso anterior. Al trabajar con el entorno de desarrollo IntelliJ IDEA, el equipo ha podido experimentar una evolución significativa en la forma de estructurar el código: de programas puramente lógicos o de consola se ha pasado al diseño de una arquitectura robusta, organizada y escalable.

La utilización del framework Spring Boot ha sido determinante para entender cómo se gestiona una aplicación en el mundo real. A través de este entorno, se han afianzado conceptos esenciales como:

- **La persistencia de datos:** Entender cómo se comunican las estructuras de código con una base de datos relacional sin necesidad de escribir consultas manuales complejas.
- **La arquitectura en capas:** Separar de forma limpia las responsabilidades del sistema (controladores, servicios y repositorios) para que el mantenimiento de la aplicación sea viable.
- **La seguridad y autenticación:** Implementar un control de accesos estricto basado en roles y tokens de usuario, garantizando que cada perfil (administrador o jugador) interactúe únicamente con las funciones que le corresponden.

Además, el proceso de desarrollo nos ha enfrentado a la necesidad de asegurar la calidad del software. La creación de pruebas de integración y pruebas de extremo a extremo (*End-to-End*) ha reforzado la importancia de la verificación automática en el ciclo de vida del software, enseñándonos a anticipar errores y a asegurar que los distintos componentes del sistema funcionen al unísono de manera armoniosa.

El Frontend y la Construcción de la Experiencia de Usuario

Por otra parte, el desarrollo del *frontend* ha permitido materializar y llevar al límite los conocimientos adquiridos durante el presente curso académico en los lenguajes fundamentales de la web: HTML, CSS y JavaScript. Si bien las primeras prácticas del año nos familiarizaron con la sintaxis y el propósito de cada una de estas tecnologías, la envergadura de este proyecto final ha exigido un nivel de comprensión mucho más profundo y maduro.

- **HTML y CSS:** Dejaron de ser herramientas para crear páginas estáticas y aisladas, convirtiéndose en los cimientos de una interfaz fluida, intuitiva y, por encima de todo, responsiva, capaz de adaptarse visualmente tanto a pantallas de ordenador como a dispositivos móviles.

- **JavaScript:** Ha jugado un papel transformador en el proyecto. Su uso riguroso ha permitido que la plataforma deje de ser un mero conjunto de formularios para convertirse en una aplicación guiada por eventos, donde la interactividad es inmediata. El manejo de la asincronía mediante peticiones en segundo plano ha sido uno de los aprendizajes más exigentes y satisfactorios, logrando que el usuario pueda consultar la disponibilidad de las pistas y realizar reservas en tiempo real sin necesidad de recargar la página por completo.

5.2. Balance Final del Proyecto

En definitiva, *ChisPadel* refleja fielmente el recorrido formativo adquirido en nuestra asignatura: Programación de Aplicaciones Telemáticas. El proyecto demuestra que no solo hemos aprendido a escribir líneas de código en diferentes lenguajes, sino que hemos desarrollado la capacidad crítica de conectar el "lado del servidor" con el "lado del cliente". También hemos logrado construir un ecosistema tecnológico completo, seguro y eficiente, cumpliendo con los estándares actuales de la ingeniería del software y consolidando una base de conocimiento que será, sin duda, el pilar de nuestros futuros proyectos profesionales.